Spare Parts Management System

Complete functions, operating guide and technical review

Prepared on 2 October 2026 | Version 1.0

A complete source-based reference for business owners, staff, administrators and developers.

Coverage	Review result
Business functions	Catalog, procurement, multi-store stock, sales, pre-orders, finance, tax, reporting and communication
Technical reference	All 325 routes and their access gates; 51 models; 17 domain services
Verification	221 PHP files syntax checked; 79 tests executed; 18 prioritized findings
Current status	76 tests passed; 3 pre-order cases need resolution; schedules are not registered

Reviewed project: spare_parts. Working-copy changes are included. Live passwords, credentials and customer data are excluded. This review does not certify production behavior or statutory compliance.

Contents

1. Review scope and system overview 3

2. Login, users, roles and permissions 5

3. Dashboard and daily overview 7

4. Products and master data 8

5. Customers and suppliers 10

6. Purchases and purchase returns 11

7. Stores, shipments and transfers 12

8. POS, invoices and quotations 13

9. Sales returns and exchanges 15

10. Vehicle pre-orders 16

11. Collections, advances and cheques 18

12. Expenses and accounting 19

13. Tax settings, invoices and ledgers 21

14. Business reports and exports 23

15. Notifications, SMS, WhatsApp and email 24

16. Settings, privacy and privileged controls 25

17. Installation and configuration 27

18. What you need to do before handover 29

19. Review findings and prioritized action plan 31

20. Validation and acceptance-test plan 35

Appendix A. Complete registered endpoint and permission inventory 37

Appendix B. Data model and relationship reference 51

Appendix C. Domain service function inventory 65

Appendix D. Source map, maintenance commands and glossary 68

1. Review scope and system overview

1.1 Purpose and audience

The application is a browser-based system for a spare-parts business. It manages products, purchasing, store inventory, counter sales, quotations, vehicle-related pre-orders, customer and supplier balances, expenses, accounting, tax records and business reports. It also includes staff permissions, printable documents, notifications and privileged visibility controls.

Owners can use the workflow and action-plan chapters. Cashiers should focus on products, customers, POS, payments and returns. Inventory staff should use purchases, shipments and store transfers. Administrators and developers should use setup, permissions, review findings and the technical appendices.

1.2 Evidence and limits

Evidence: application routes, controllers, services, models, Blade views, migrations, seeders, dependency manifests, existing documentation and automated tests. The route inventory was obtained from Laravel itself. Source references are repository-relative paths and method names; the project root is /Applications/XAMPP/xamppfiles/htdocs/spare_parts.

Snapshot: Git commit fe76369 plus the current local changes. Inventory: 325 registered routes, 37 controller PHP files, 51 model files, 17 service files, 67 migration files, 149 Blade view files, 121 distinct permission keys in registered routes and 12 PHP test files. Controller file counts include the base controller, a recovered report copy and an empty legacy controller.

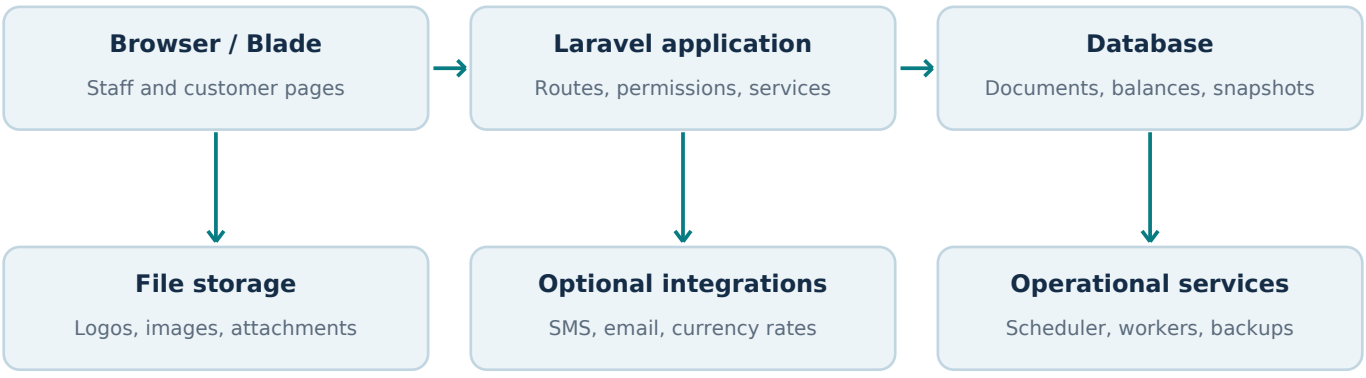
Validation: 221 PHP files passed syntax checks. PHPUnit executed 79 tests with 349 assertions: 76 tests passed, two failed and one errored. All three problems were in pre-order stock behavior. The scheduler reported that no scheduled tasks were defined.

This is a source review with isolated automated checks. Live database contents, production migrations, browser interaction, printers, barcode scanners, server configuration, external SMS delivery, email delivery and real bank operations were not exercised. A registered route is not proof that its controller action is implemented or operational.

1.3 Functional map

Area	Functions present
Access and administration	Username/email login, logout, users, roles, store assignments, activity logs
Product catalog	Products, category trees, brands, units, price options, imports, barcodes, stock alerts, write-offs
Procurement	Supplier records, purchases, shipping costs, attachments, supplier payments, purchase returns
Inventory	Stores, default store, shipment GRNs, landed costs, allocations, transfers, transfer history
Selling	POS cart, discounts, unit selection, held carts, checkout, split payments, invoices, quotations, returns and exchanges
Pre-orders	Vehicle details and images, quoted items, inventory linking, prices, deposits, completion, cancellation, reopening, PDFs and reports
Customer balances	Opening balances, invoice collections, payment allocation, advance credit, statements, reminders, public bill/history pages
Finance	Chart of accounts, cash/bank transactions, petty cash, bank reconciliation, owner capital/drawings and accounting statements
Tax	Versioned settings, line calculations, ledgers, invoices, payments, adjustments and 14 report types
Reports and communication	Sales/stock/profit/due reports, exports, rate conversion, notifications, SMS and WhatsApp links
Configuration and privileged controls	Business/print/POS settings, privacy mode, super-admin display controls and information-page controls

1.4 Architecture



The browser renders Laravel Blade pages and uses AJAX for POS, quick creation and other interactions. Laravel routes apply session authentication and permission middleware. Controllers coordinate Eloquent models and domain services. The database holds business records, financial balances, tax snapshots and configuration. Uploaded files are stored separately and must be backed up with the database.

External integrations are optional: SMS uses configured HTTP providers, tax invoice email uses Laravel mail, exchange-rate pages call external services, and WhatsApp creates links for an operator to open.

Sources: composer.json; package.json; bootstrap/app.php; app/Providers/AppServiceProvider.php; routes/web.php.

2. Login, users, roles and permissions

2.1 Login and logout

Staff open /login and enter a username or email with a password. The controller selects email authentication when the input is an email address, otherwise username; it falls back to the name column on older schemas without username. Where available, the user must be active. A successful login regenerates the session and writes an activity record. Super Admin users go to /fun/dashboard; other users go to /dashboard.

Logout is a POST action. It records the event, invalidates the session and regenerates the CSRF token. A remember-me option is present. No password-reset, self-registration or multi-factor workflow was found in the application routes. No login-specific throttle is declared on the login route.

2.2 User management

Authorized staff can list users, create accounts, edit profile details, reset a user's password through editing, assign a role, change active status and assign stores. The controller protects the last active administrator and prevents routine removal of the currently logged-in account. Super Admin users and roles are hidden from ordinary administrators in user/role management.

2.3 Role management

Roles store a name, description, active flag and an array of permission keys. Ordinary roles can be created, viewed, edited and deleted subject to controller restrictions and their user assignments. Reserved administrator role names are protected. Permission selection spans the catalog, POS, pre-orders, accounting, stores, tax, reports, settings and communication modules.

Admin and Super Admin are special: User::hasPermission and Role::hasPermission grant them all permissions by role name, independent of the stored array. The /fun routes additionally require Super Admin specifically. Normal Manager and Cashier access comes from actual stored permissions, which may differ from seed defaults.

Role	Intended operational use	Important behavior
Super Admin	System owner and privileged configuration	All permission checks pass; separate privileged dashboard
Admin	General business administration	All permission checks pass; special display controls can still affect this role
Manager	Inventory, purchasing, sales and selected finance	Granular stored permission list; verify each accounting/store subsection
Cashier	POS, customer creation and selected pre-order tasks	Completion, price/cost visibility, editing and cheque actions depend on separate permissions
Custom role	A business-defined job	Grant the minimum operations needed and test direct route access

Store assignments are represented by the store_user relationship and User::hasStoreAccess. Their enforcement is not uniform across controllers; assignments should not yet be treated as a verified isolation boundary.

2.4 Permission conventions

Most modules use dotted keys such as products.view and purchases.edit. Product price actions use keys such as view_product_prices. Pre-orders use keys such as preorder_complete. Seeing a menu entry does not establish permission for the corresponding action. Appendix A records the actual route permission for every endpoint.

Privacy toggling is validated inside PrivacyModeController/PrivacyModeService. The storage-link helper checks settings.edit inside its closure. Customer PDF exports, category child lookup and bank system-balance lookup have authentication but no module-specific route permission; their controller behavior differs and needs review.

Sources: Auth/LoginController.php; UserController.php; RoleController.php; app/Models/User.php; app/Models/Role.php; app/Http/Middleware/RequirePermission.php; EnsureSuperAdmin.php.

3. Dashboard and daily overview

The dashboard supports a date interval and defaults to today when the interval is incomplete. It calculates sales, purchases, expenses, sales profit, net profit, comparisons with an earlier period, customer and supplier amounts, stock information, recent activity and chart data. Figures are affected by the configured hidden records/ranges and display controls.

The dashboard calculates cost of goods using the product's current cost price and returned quantities. Changing a product cost can therefore alter historical profit output. The displayed total sales can also be overridden by privileged settings. Staff should reconcile financial reports using an unrestricted view and known filters.

Opening the dashboard calls ChequePaymentService::autoPassEligible. When enabled, eligible pending cheques can be marked passed during this page request. This is a real update, not merely a visual reminder. The cheque list also invokes automatic processing. The operation should be configured and approved as part of the business's cheque policy.

Daily use: choose the interval, check low-stock and cheque reminders, review due amounts, compare sales and expenses, and investigate unexpected values using the underlying document lists. Confirm the viewing role and privacy/visibility state before using totals for reconciliation.

Sources: DashboardController::index; ChequePaymentController::index; DashboardVisibilityService.

4. Products and master data

4.1 Categories, brands and units

Categories support parent/child relationships, trees and category filtering; products can belong to multiple categories through a pivot table while retaining a legacy primary category. Brands also support a pivot relationship and a legacy primary brand. Units include a short name and base-unit multiplier. Authorized users maintain these lists and their active flags. Several standalone detail/create views are missing; use the implemented list/inline forms and see R14.

Create categories, brands and units before importing or entering products. Use consistent unit names and decide which unit represents stock. Verify conversion factors with real pack sizes before selling by alternative units.

4.2 Product records and search

Product creation and editing support name, SKU, barcode, description, image, category/brand assignments, unit, visible units, cost, selling price, quantity, stock alert quantity and active state. Product screens also support store quantities and store exclusions. Pre-order products can be listed separately using the is_pre_order flag.

The product list supports search by name, SKU and barcode, main/subcategory and brand filters, low-stock/out-of-stock filters and a store selection. The details page shows purchases, sales, stock transfers and activity records as a timeline. Direct price updating has its own permission.

4.3 Price options and cost visibility

A product can have several price options, each with cost, selling price, stock quantity, active/inactive status and a default flag. Duplicate active cost/selling-price pairs are rejected. Setting an active default clears other defaults and updates the product's fallback cost/selling price. Removing a price used in a sale makes it inactive instead of deleting it; other relationship cases need validation.

The POS can use price-wise stock. Cost visibility in the price popup requires the setting show_cost_price_in_pos_popup and the permission view_cost_price_in_pos. Enable price-wise stock only after product-level, price-level and store-level balances have been reconciled.

4.4 Unit conversion

Product helpers convert prices using unit_factors and base_unit. POS item units can change displayed price and required stock. Checkout deducts quantity multiplied by at least 1.0; factors below 1.0 are not handled symmetrically. Product stock is cast/stored as an integer while store and price quantities allow decimals. Fractional stock and sub-unit sales require acceptance testing and schema/logic alignment.

4.5 Spreadsheet import

Download the template from Products > Import. Supported upload types are XLSX, XLS and CSV. The template contains Name, SKU, Category Names, Brand Name, Unit Short Name, Cost Price, Cost Code, Selling Price, Profit Margin %, Profit Margin Fixed, Stock Quantity, Alert Quantity and Description.

Name and a recognized unit are required. A blank SKU is generated; a duplicate SKU is skipped. Category names may be separated with a comma or pipe. Unknown categories or brands produce notices and may leave the record only partly mapped. A valid cost code overrides numeric cost price. Profit-margin columns are template formulas rather than separately stored imported attributes. The importer reports imported count and row issues.

Import small batches first. The importer creates product-level stock but does not establish store/price stock in the same way as normal stock workflows. Reconcile all three levels after import before POS use. There is no whole-file atomic transaction or import undo workflow in the reviewed controller.

4.6 Barcode printing

Staff can search/select products, specify copies and generate a label preview. Barcode settings include label/paper width and height, labels per row, spacing, alignment, margins, barcode height, text sizes, top/bottom padding, encoded cost display, encoded selling-price display and named presets. Some encoded-price controls are restricted to Super Admin.

Print a small batch at 100% scale and check scanner readability and physical label alignment before bulk printing. Encoded price text is a display convention; it is not encryption.

4.7 Alerts and write-offs

POS stock deduction invokes StockAlertService when product quantity is at or below its alert threshold. It creates internal notifications for users with the matching admin role name, with a first-user fallback. This service does not automatically send SMS and does not use every stock-alert setting shown on the notification settings page.

Write-off records require product, integer quantity, reason and date. The controller checks total stock, deducts product quantity, calculates cost using current product cost, records an expense in Product Write-off, checks the category limit and records activity. It does not adjust store/price quantities or call the normal expense-accounting service. See finding R06.

Sources: ProductController; ProductPriceController; CategoryController; BrandController; UnitController; ProductWriteOffController; ProductImportTemplateExport; app/Models/Product.php; StockAlertService.

5. Customers and suppliers

5.1 Customer records

Customers have name, TIN, contact information, address, city, country, opening balance and active status. Name-only creation is supported; phone/email may be absent. A supplied customer TIN is validated as 9-12 digits and unique. Quick customer creation is available to authorized pre-order/POS users through the relevant interactions.

Customer details combine sales, payments and pre-orders with balance summaries. The model separates sales due, pending pre-order due and available advance. Sales due is calculated from invoice due plus opening balance minus generic payments that are linked to neither a sale nor a pre-order. When that net amount is negative, its absolute value becomes the advance balance.

5.2 Statements and customer PDFs

Customer exports include sales, ledger/all transactions, payments-only and detailed pre-orders. A date interval can be supplied; the controller defaults to the current calendar year. Printed figures and invoice references can be affected by visibility/privacy controls. PDF export permission is not separately enforced at the route level.

The customer can receive a single-bill or total-due reminder through SMS/WhatsApp interactions. Authorized users can add generic advances, allocate payments to opening balances/invoices and later apply available advance at checkout.

5.3 Public customer pages

The system exposes /customer/{customer}/bill/{sale}, /customer/{customer}/history and /customer/{customer}/pay/{sale} without staff login. Bill/payment handlers check that the sale belongs to the supplied customer and apply relevant visibility checks. These links use numeric IDs without a signed access token, so knowing or discovering valid IDs can expose customer information.

The public payment page tells customers to contact the business to arrange payment. It is not an online payment gateway, checkout provider or bank API. Payments still need to be recorded by staff.

5.4 Supplier records and balances

Suppliers have name, company name, TIN, contact details, opening balance and active status. Supplier name and phone are required by the controller. The supplier details page presents purchases and payment information. Authorized staff can allocate payments to individual purchases and opening balances.

Supplier due is the sum of purchase due plus opening balance minus generic payments with no purchase link. Its computed value is not clamped to zero in the model. Negative amounts require an explicit business interpretation, such as supplier credit.

Sources: CustomerController; SupplierController; BulkPaymentController; PaymentController; app/Models/Customer.php; app/Models/Supplier.php; customers/public-payment.blade.php.

6. Purchases and purchase returns

6.1 Purchase creation

Select the supplier, date, reference, status and destination stores. Add products with quantity, unit cost, selling price, optional line discount and store allocations. Whole-bill discounts may be fixed or percentage. Supported statuses are pending, ordered and received. Optional supplier tax invoice details and input-VAT claimability are recorded.

Purchase documents can include PDF, JPG/JPEG/PNG, CSV, ZIP, DOC or DOCX attachments up to 5 MB. Shipping can be classified as divided into item costs or expense. Divided shipping is apportioned by total quantity and added to the calculated inventory unit cost. Non-claimable VAT is included in inventory value; claimable input VAT is treated separately by the tax service.

The controller supports a payments array and creates/uses price options. Store allocation quantities are supplied per item; a default-store fallback is used when missing. Ordinary purchase creation immediately increases product stock and applicable price/store stock, even when status is pending/ordered or the is_pre_order flag is set. Status is not a verified stock-receipt gate.

6.2 Purchase payments and editing

The creation/edit logic supports ordinary methods, endorsing a pending customer cheque to a supplier and issuing an own cheque. Own cheques are tracked in held_own_cheque_amount. Purchase due is generally total minus paid and held own cheques.

The edit action changes supplier/reference/date/status, appends supplied payments to stored paid/held totals, replaces an attachment if supplied and updates notes. It does not re-enter purchase item quantities/prices. Ordinary creation/edit payments update document totals without creating all equivalent Payment/accounting rows. Use the supplier payment screens with careful reconciliation until the workflows are unified.

Deleting a purchase reverses tax posting and deletes the purchase; the reviewed destroy action does not reverse previously added stock or all financial effects. Deletion must not be used as a routine correction process for received purchases.

6.3 Purchase return workflow

Find a purchase using document ID/number, supplier or date. Select item quantities and choose cash or account. The return records original item cost, reduces product quantity, creates a tax return adjustment and writes an activity record. Account returns decrease purchase due; cash returns do not change that due.

The controller checks the original purchased quantity and available product stock, but does not aggregate earlier returns to calculate remaining returnable quantity. Store/price stock, payment status and cash/accounting postings are not fully synchronized. Repeated returns and returns against partly paid purchases need correction before general use.

Only purchase-return listing, creation and storage are implemented. The resource registration also advertises show/edit/update/delete actions without controller methods; these are incomplete endpoints.

Sources: PurchaseController::store/update/destroy; PurchaseReturnController; TaxPostingService; TaxReturnService.

7. Stores, shipments and transfers

7.1 Store management

Create stores with name, unique code, phone and address. Select the default store. Separate permissions exist for the overview, store list, shipments, allocations, transfers, stock report and transfer report; writes use stores.manage. The user/store assignment feature is described in Chapter 2.

7.2 Shipment GRN and landed cost

Record a shipment number, unique GRN number, optional supplier, shipment/received dates, freight, duty, other costs and notes. Items require a product, positive decimal quantity and non-negative unit cost; a selling price may be entered.

Extra costs are distributed in proportion to item base value. Landed unit cost equals the item unit cost plus its share of extra costs divided by quantity. Shipment creation stores the shipment and items. Allocation creates a relationship between a shipment item and destination store and prevents allocating more than the original shipment quantity.

The allocation path adds store stock only; it does not add product-level or price-level stock. Treat shipments/allocations and purchases as separate implemented mechanisms until a reconciled receiving design prevents missing stock or double receipt.

7.3 Store transfer

Choose different source/destination stores, date, reference and one or more products/quantities. Optional shipping and additional expenses are stored on the transfer. Duplicate product rows are grouped. The controller checks aggregate source stock, deducts source rows and adds destination stock.

Transfers have a header and item records. The history supports from/to date, source/destination store and reference/product search. Transfers can be edited or deleted; these actions revert earlier store movement before applying new movement or deleting the record.

Transfer deduction can use price-specific source rows, while destination additions use a generic row with no price option. Transfer reversals may reduce destination stock below zero if that stock was sold later. Movement charges are stored as transfer fields rather than automatically becoming normal expense/accounting entries. Confirm these rules before enabling unrestricted edits/deletions.

7.4 Inventory balance model

There are three balances: product.stock_quantity, product_prices.stock_qty and store_stocks.quantity. Sales, purchases, returns, pre-orders, imports, write-offs and shipments do not update them uniformly. A store transfer should move branch availability without changing total owned stock, while receipt/return/write-off affects ownership totals. The current system needs a single documented rule for all three balances.

Sources: InventoryStoreController; app/Models/Store.php; StoreStock.php; StoreTransfer.php; StoreStockTransfer.php; StockShipment*.php.

8. POS, invoices and quotations

8.1 Cart operations

Open /pos, select the store and search by name/SKU/barcode. Choose the product and, where enabled, its active price option. Add quantities, change units, set line discounts/notes, set a whole-bill discount, remove items and clear the cart. The cart is stored in the browser's server session.

Held carts can be labelled, listed, loaded and removed. They are session data, not permanent shared orders. Losing a session can lose cart/hold information. Save a quotation or pre-order when a durable document is needed.

Recent sales and sale search are available. Original sale items can be added as negative return lines to perform a return/exchange through POS. Return quantity is checked against prior returns.

8.2 Checkout

1. Confirm products, price options, unit quantities, discounts and branch availability.
2. Select a customer when selling on credit, accepting a cheque or issuing a tax bill.
3. Choose normal or tax bill. Tax bills require tax.invoice.print and complete tax/customer details.
4. Enter a single payment or split payments. Supported methods are cash, credit, cheque, bank deposit, bank transfer, card and mobile payment.
5. Add cheque number/date and optional bank/account details when applicable. Separate cheque-creation permission is required.
6. Optionally apply available customer advance. Confirm paid, held cheque, due and tendered amounts.
7. Complete the transaction and print the receipt/invoice. Recheck stock, payments and the customer's balance.

Checkout uses a database transaction, checks available stock and recalculates taxes. It writes sale/item records, payments/cheque holds, tax records and selected accounting entries; reduces applicable stock; and clears the cart after commit. Return-only checkout can return a return ID without a new sale ID.

8.3 Amounts and statuses

The principal sale balance is due = max(0, total - paid - held cheque). A cheque held pending clearance is separate from money actually received. Payment status remains partial while a held cheque exists, even if visible due is zero. Tendered amount records money presented before change; it should not be confused with net applied payment.

Card fees can be added to the customer total or absorbed by the seller. A seller-paid card fee can create an expense if enabled and a category is selected. Its expense-accounting synchronization needs attention because this path creates Expense directly.

8.4 Invoice management

Sales can be listed, filtered, viewed, printed, exported as CSV/PDF, edited via POS and deleted subject to permissions. Details show items, payments, due and returned quantities. Add-payment accepts later collections. Regular invoice paper options include A4, Letter, 80 mm and 58 mm in settings, with logo, footer and terms.

Sale numbers use an encoded daily sequence and random suffix with INV or QUO prefixes. Official tax invoice numbers use a different service. Changing an invoice prefix setting should be tested; regular numbering is not simply a concatenation of the settings-screen prefix.

Editing an existing sale restores old product/price stock and replaces sale items/tax lines, then applies new stock. It does not consistently restore old store quantities. Checkout accepts edit_sale_id on a pos.access route

without repeating sales.edit authorization. Deleting a sale restores product/price stock, considers returns and reverses tax, but requires broader store/accounting reversal review.

8.5 Quotations

Create a quotation from the POS quotation screen or save a draft. Quotations use sale_type=quotation, have QUO numbers and printable PDFs with validity, logo, terms and footer. They do not deduct stock at draft save. Authorized users can list and convert quotations into sales; conversion checks stock and posts tax. It does not mirror every store/payment/accounting step of normal checkout.

Sources: POSController; SaleController; SalePaymentAccountingService; SaleRecalculationService; sales/print.blade.php and partials; quotations/pdf.blade.php.

9. Sales returns and exchanges

Find the original sale, choose return quantities and select cash or account refund. The dedicated return module can also add replacement products. Return quantities cannot exceed sold quantity minus earlier returns. The process records return items, restores product/price stock, creates tax adjustments and recalculates the original sale.

For exchanges, the module can create a linked replacement sale and apply the return value toward it. Depending on the net difference, it records refund, account credit or return-adjustment payments. A return print view is available. Staff should verify both the original invoice and replacement sale after an exchange.

The dedicated return controller and the inline/POS return paths differ. POS returns restore original store stock; the dedicated SaleReturnController and SaleController::returnSale do not perform an equivalent store-stock restoration. Dedicated cash/account refund Payment rows do not consistently call the normal payment accounting service. Return/exchange stock and cashbook effects therefore need end-to-end acceptance checks.

Supported dedicated actions are list, create, store and print. Show/edit/update/delete resource actions are registered but missing. There is no verified general-purpose reversal interface for an existing sale-return document.

Sources: SaleReturnController; SaleController::returnSale; POSController::addReturnItem/checkout; TaxReturnService; SaleRecalculationService.

10. Vehicle pre-orders

10.1 Record and document fields

A pre-order requires customer, date, document type, vehicle name and at least one item. Optional data include store, vehicle description/image, instructions, notes, expected delivery date, custom tax rate, tax display mode and bill discount. Images accept JPG/JPEG/PNG/WebP up to 5 MB. Delivery date cannot precede order date.

Each item can reference a product/price option or remain temporary/unlinked. It has original product name, description, integer quantity, quoted/final unit prices, fixed/percentage discount and notes. Percentage discounts cannot exceed 100%. A selected price must belong to the selected product.

10.2 Current stock behavior

Saving a pending order deducts stock immediately for linked items. The service may also automatically link an active product with the same name. A zero-stock linked product currently prevents saving. Editing restores old linked-item stock and then deducts the new linked items. Manual linking deducts stock; unlinking restores it. Completion does not deduct the same linked stock again.

This differs from the existing test contract, which expects pending orders to leave stock unchanged and allow zero-stock products. The three failing/error tests are evidence of this unresolved policy change. The documentation describes the working copy, not the earlier test expectation.

Completion can automatically create active zero-cost, zero-stock products for still-unlinked items and then generate sale items without a stock deduction for those newly created products. Linking the same item repeatedly can deduct it repeatedly because the linking routine does not first reverse its earlier allocation. These need correction before relying on pre-orders for reservation control.

10.3 Available actions

Action	Implemented behavior
List/search/filter	Pending/completed/cancelled status routes, customer and other filters
Create/edit	Save customer, vehicle and items; only pending orders are editable
Link/unlink product	Associate an item with inventory or remove its link; affects stock in the current copy
Change price	Keep quote, use current selling price or specify custom price; updates totals
Add payment	Cash/bank/card/mobile/cheque interactions; tracks due and payment history
Delete payment	Checks ownership; restricts passed-cheque removal to cheque management
Complete	Creates a normal sale, item/tax/payment records and completion metadata
Cancel	Pending only; records actor/time/reason and restores linked-item stock
Reopen cancelled	Returns to pending and deducts linked-item stock again
Reopen completed	Reverses linked sale payment/tax effects, restores stock, deletes sale, returns to pending and deducts stock again; blocks when linked returns exist
Print/export	Quotation and invoice PDFs, order/customer reports and detailed customer PDFs
Quick customer/product search	Supports creation and selection from the order workflow

10.4 Deposits and completion

Pending-order payments can be collected before completion, including held cheques. Completion combines stored deposits with newly submitted payments. Pending deposits and cheques are not fully migrated to the resulting sale in the completion routine, and the payment cap validates new payments against grand total rather than remaining due. A pending edit also resets paid/held totals while existing payment records remain. See R03 for the financial consequences.

Use a controlled workflow until corrected: confirm all links and physical availability, record payments once, review payment history before editing/completing, and reconcile both pre-order and linked sale. Status changes and price/link operations require separate permissions. Pre-order activities retain action details and old/new values.

Sources: SavePreOrderRequest; PreOrderController; PreOrderService; PreOrderImageService; app/Models/PreOrder*.php; tests/Feature/PreOrderManagementTest.php.

11. Collections, advances and cheques

11.1 Customer and supplier collections

Customer bulk-payment screens display unpaid invoices and opening balance due, with allocations to either. Supplier bulk payments similarly allocate to purchases or opening balance. Individual purchase payments are available through PaymentController; invoice payments through SaleController. Payment date, amount, method and notes/reference are recorded where the respective form supports them.

Bulk routines iterate allocations without a single encompassing transaction, do not consistently cap against remaining balances and accept a free-form method. A partial failure may leave partial results. Customer bulk cheque allocation immediately creates a Payment and reduces invoice due before creating a pending cheque, unlike POS cheque holding. This divergence must be corrected before using the cheque method in bulk collections.

Advances are generic customer payments with no sale link. POS can apply available advance to an outstanding sale and records an advance-deduction entry. Advance activity is separate from pending pre-order deposits; verify statement calculations do not mix those records.

11.2 Cheque management

Cheque records include customer/sale or supplier/purchase references, type, date, number, bank/account name, amount, status, user and processing metadata. The cheque screen filters records and supports pass/return. Passing an ordinary sale-linked pending cheque creates a Payment, reduces held amount, increases paid amount, updates due/status and posts accounting. Returning a held cheque removes the hold and makes the amount due again.

Passing can deposit to a selected bank or endorse to a supplier. Automatic passing is enabled by a setting and uses cheque date plus a configurable grace period, defaulting in code to 15 days. Dashboard and cheque listing invoke this automatically, even while scheduled tasks are not registered.

Own purchase cheques, opening-balance cheques and pending pre-order cheques may have no sale_id. The current pass/return service assumes a sale exists, so these cheque types cannot be described as a complete working clearance lifecycle. The automatic query also does not exclude them.

Sources: BulkPaymentController; PaymentController; SaleController::addPayment; CustomerController::storeAdvance; ChequePaymentController; ChequePaymentService; SalePaymentAccountingService; BulkPaymentAccountingService.

12. Expenses and accounting

12.1 Expense records and categories

Authorized users create, view, edit and delete expenses with category, optional store, date, amount, method, description and receipt. Methods are cash, bank transfer and petty cash. Receipt files accept JPG/JPEG/PNG/PDF up to 2 MB. Categories support limits, reset frequency and a monthly reset day. Expense entry/edit checks category spending against the configured limit.

Normal expense entry synchronizes accounting through ExpenseAccountingService. Updating reverses earlier postings before applying new ones; deleting reverses accounting and removes the receipt. Ordinary petty-cash expenses use the first active fund rather than a selectable fund. The reversal uses the expense's current method, so editing from petty cash to another method needs careful fund-balance testing.

Write-offs and seller-paid card fees create Expense directly without the same accounting sync. Petty-cash module expenses use PettyCashExpense rather than Expense and are not automatically included in ordinary expense/profit-loss reports.

12.2 Chart of accounts and transactions

Create accounts with unique code, name, type, subtype and opening balance. Types are asset, liability, equity, income and expense. Transactions have account, optional related account, date, in/out direction, method, amount, reference, description and optional bank/cheque information. Manual transactions update selected balances; a related account receives the opposite direction.

System services use codes including 1100 cash, 1200 bank, 1300 customer receivable, 4000 sales revenue and 5000 expense, with petty cash accounts and tax accounts also present. This is custom accounting functionality described in the code as QuickBooks-style. No QuickBooks API integration was found.

Accounting statements mix account balances and recorded transactions. Sales/payment/tax services do not constitute a verified complete accrual posting system for every transaction. Reports must be reconciled before they are treated as final accounts.

12.3 Banks and reconciliation

Create a bank record with bank name, account name/number and opening balance. The system uses a shared bank chart account and separately records bank IDs for some transactions. Reconciliation selects a statement month, statement balance and optional transaction IDs. It calculates the system balance at month end and stores the difference as tallied/difference status, notes and actor/date.

The system-balance calculation uses bank-linked movements when found and otherwise falls back to movements with no bank ID against the shared chart account. Many automated postings omit bank_account_id, so multiple-bank statements require a dedicated reconciliation test and consistent tagging.

12.4 Petty cash

Create funds with opening/current balance and a linked chart account. Record fund-specific expenses with category, voucher, date, amount and description. Fund expense entry checks available balance and writes two accounting movements. Transfer cash in either direction between the main account and a selected fund; this updates both accounts and fund balance. The transfer routine does not explicitly reject a transfer exceeding source cash.

12.5 Owner equity

Authorized users create, edit and delete owner capital/drawing records with date, amount, payment account, reference and notes. The controller posts asset/equity movements and reverses earlier movements on update/delete. Separate view/create/edit/delete permissions apply.

12.6 Accounting screens and exports

Screens include overview, accounts, transactions, banks, petty cash, ledger, T-accounts, trial balance, balance sheet, cashbook, bankbook and owner equity. The registered export endpoint supports PDF or Excel for accounts, transactions, banks, petty cash, ledger and cashbook. It does not offer every screen as an export section.

Sources: ExpenseController; ExpenseCategoryController; Accounting/AccountingController; Accounting/PettyCashTransferController; ExpenseAccountingService; app/Models/Accounting/*.

13. Tax settings, invoices and ledgers

13.1 Configurable tax behavior

General settings expose VAT enabled/registered flags, business TIN, rate, sale/purchase inclusive or exclusive modes, customer invoice VAT display, product override, official invoice prefix/branch/sequence, effective date and template version. Settings changes create effective-dated versions. Products can override tax status/rate and price mode when allowed. Tax statuses include standard, zero-rated and exempt.

The service uses scaled decimal arithmetic, applies line/bill discounts before VAT, calculates taxable values and VAT, and stores transaction-line snapshots. Stored snapshots protect historical documents from later setting changes. These are software functions; the review does not certify current tax law, statutory rates or invoice compliance.

13.2 Official tax invoices

An official tax invoice is available only for a completed sale meeting the service's checks: enabled/registered VAT, business TIN/name/address/phone, purchaser TIN/name/address/phone, invoice date, standard-taxable VAT and reconciled stored totals. A walk-in customer cannot checkout with a tax bill.

The number service locks the relevant tax settings row, allocates a unique prefix/branch/sequence number, prevents whitespace/duplicates and reuses an already issued number. The tax invoice can be downloaded/printed and emailed as a PDF attachment. Email requires working mail configuration. Invoice totals come from database-backed tax records rather than a fresh generic rate estimate.

13.3 Input/output ledgers and returns

Output entries arise from sales and input entries from purchases; purchase claimability affects VAT receivable. Posting includes transaction/line references and accounting effects. Repeated posting is intended to be idempotent. Returns use original tax snapshots for negative adjustments. Reversal retains tax ledger/accounting reversal entries.

13.4 VAT payments and adjustments

VAT payments have period, date, unique reference, method, account, amount, notes and optional PDF/image attachment up to 5 MB. Drafts can be edited or deleted; finalization posts accounting. Finalized payments can be reversed with a retained audit trail. Period-balance calculations include prior balances and payments. Review overpayment/credit handling with the accountant before live use.

Adjustments have period, date, debit/credit type, amount, reason, reference and draft/approved state. They can be created and approved. No general edit/delete/reversal route for adjustments is registered.

13.5 Tax report catalog

Report	Purpose
Output VAT	Tax generated by sales
Input VAT	Purchase-side tax and claimability
VAT Summary	Period totals and balances
VAT Payable	Outstanding VAT calculation
VAT Payment	Recorded tax payments
VAT Adjustment	Debit/credit adjustments
Taxable Sales	Standard-taxable sales lines
Taxable Purchase	Taxable purchase lines
Zero-Rated Supply	Zero-rated lines
Exempt Supply	Exempt lines
Customer VAT Invoice Register	Customer invoice register
Supplier VAT Invoice Register	Supplier invoice register
Sales Return VAT	Sales return tax effects
Purchase Return VAT	Purchase return tax effects

Reports support date/preset and applicable party/store/product/category/user filters. PDF and XLSX use the filtered rows and totals. Tax-specific permissions cover viewing, payments, adjustments, report viewing/export and invoice printing.

The older /reports/vat screen estimates inclusive/exclusive VAT using current generic settings and sales totals. It is distinct from /tax reports based on stored tax lines. The two should not be assumed equivalent.

Sources: TaxManagementController; TaxCalculationService; DecimalMath; TaxPostingService; TaxReturnService; TaxInvoiceNumberService; TaxReportExport; SettingController.

14. Business reports and exports

Report/screen	Contents and use	Registered export
Sales	Sales documents, totals and applicable date/customer/store/product filters	PDF, CSV
Daily ledger	Daily selling and collection information	PDF, CSV
Purchase	Purchase totals, supplier and category/store filters	PDF, CSV
Profit and loss	Revenue, current-cost-based COGS, gross profit, expenses, net profit	PDF, CSV
Stock	Quantity, cost/selling value, low stock and catalog/store filters	PDF, CSV
Expense	Expenses, category summaries and date/store filters	PDF, CSV
Trending	Best-selling products with return-adjusted quantities/revenue	Screen
Legacy VAT	Current-rate inclusive/exclusive estimates and daily details	PDF, CSV; day PDF
Receive	Customer/sale-linked incoming payments and daily totals	PDF, CSV
Debit	Supplier/purchase-linked outgoing payments and daily totals	PDF, CSV
Due bills	Outstanding sales invoices	PDF, CSV
Customer due	Due aggregated by customer	PDF, CSV
Never sold	Products without sale history	Screen
Unsold recently	Products not sold during a configurable number of months	Screen
Rate conversion	Currency rate/conversion with external fetch and manual rate	Screen
Store stock	Branch stock overview	Screen
Transfer history	Transfer headers/items, date/store/reference search	Screen
Pre-order reports	Order/customer summaries, statuses, amounts and items	PDF paths in controller

Report filter names/defaults vary. Date intervals, store selection, hidden data settings and privacy mode can materially change the output. Export and screen code are sometimes separate implementations. Select the same filters and compare totals during acceptance testing.

Profit-loss COGS uses the current product cost, not a historical item cost snapshot. It does not explicitly exclude quotations in its initial sale query. Historical profits can change and quotation data can enter the calculation. PettyCashExpense records are separate from the normal expense query. This report needs correction before being used as an authoritative profit statement.

Rate conversion tries several HTTP services, caches results and accepts a manually stored rate. It is an informational tool; it does not automatically revalue inventory or invoices. The HTTP fetches disable certificate verification and should be corrected.

Sources: ReportController; AccountingController::export; InventoryStoreController::transferHistory; PreOrderController::report/customerReport.

15. Notifications, SMS, WhatsApp and email

Internal notifications show recent items for the current user, an unread count and a mark-read action. Promotion history is also available. Stock alerts create internal notifications during supported stock operations.

Notification settings include provider selection, enable flags, credentials, sender information, custom endpoint, stock thresholds and monthly due-reminder settings. SMS providers exposed in code are custom/TextIt-style, Twilio, Nexmo and MSG91. SmsService actually gates sending using enable_sms_notifications; do not assume the separate sms_enabled flag alone activates delivery. Some provider-labelled endpoints need verification before those provider options can be considered working.

Promotion sending supports SMS, WhatsApp or both, a message and all/active/selected customers. The form also offers Top 5 Spenders, but controller validation does not accept top5, making that option incomplete. Phone numbers are filtered/deduplicated for bulk sending. Send counts and WhatsApp links are stored in a notification record.

WhatsApp functions generate WhatsApp Web links with phone/message text. The operator must open the link and send the message. Counts labelled WhatsApp sent are generated-link counts, not delivery confirmation. There is no automatic WhatsApp Business API delivery in the reviewed service.

Monthly reminders have enabled flag, day 1-31, time, channel, template and excluded customers. Templates replace {name}, {due} and {date}. A manual send-now action is present. The CLI command checks the configured day and sends/creates links for customers with invoice due and a phone. A day such as 31 will not run in shorter months unless a last-day fallback is added. No repeat-send protection is evident.

The intended reminders and cheque auto-pass schedules are in app/Console/Kernel.php, but the bootstrapped application reports no scheduled tasks. Cron alone will not resolve this; scheduling must be wired into the active Laravel configuration and verified. Tax invoice email is separate and depends on Laravel mail settings.

Sources: NotificationController; SmsService; WhatsAppService; StockAlertService; SendMonthlyDueReminders; AutoPassChequePayments; app/Console/Kernel.php; routes/console.php.

16. Settings, privacy and privileged controls

16.1 Settings checklist

Settings group	Configurable functions
Business	Name, tagline, address, phone, email and logo
General	Currency position/decimals, date/time, timezone, language, pagination, low-stock flag and tax settings
Invoice	Prefix field, paper size, logo, footer and terms
Quotation	Prefix field, validity days, paper size, logo, footer and terms
Pre-order	PDF line/text/heading colors and logo shape
POS	Layout, default store, price-wise stock, cost popup, card fees and cheque reminders/auto-pass
Barcode	Dimensions, spacing, fonts, barcode height, encoded prices, code maps and presets
Notifications	Provider/configuration, stock flags, monthly reminders, exclusions and templates

Some generic settings are not uniformly consumed by all views/controllers. Language is stored but a complete translation system was not established from the routes/views. Paper/prefix choices require output testing. Currency/timezone are shared by AppServiceProvider, with an Asia/Colombo fallback.

16.2 Privacy mode

Super Admin configures privacy enablement, shortcut keys, invoice limits, masking style and the pages to which it applies. The session toggle requires privacy_mode.toggle and rejects users with bypass permission. As Admin and Super Admin automatically pass all permissions, they receive bypass behavior from this check. Shortcuts default to Alt+S and Cmd+X in code and can be configured.

Masking modes include hidden values, asterisks and a reduced amount. On selected pages, invoice references are replaced with daily ordered labels. Sales-list, report, dashboard, POS recent-sale and customer-history behavior differ. Customer/history prices may remain visible while invoice references change. Privacy toggles are logged with user, action, page, IP and time.

Privacy mode changes presentation for applicable session/pages. It is not a substitute for authorization or encryption. It should be disabled or bypassed during record reconciliation.

16.3 Super Admin dashboard

The separate /fun/dashboard has overview, dashboard controls, product/stock controls, hidden-record controls and privacy sections. It can select hidden products, suppliers, customers, sales and stores, apply amount ranges, restrict areas and scale displayed values/profit/customer amounts. Search endpoints support selection. Ordinary administrators can receive modified views; Super Admin sees the privileged perspective.

16.4 Information-page controls

The public /information page presents a business visiting card. A code login opens a separate session-protected settings page that can override dashboard sales amount/count, hide sale/purchase amount ranges and enable a forced error mode. A range-bills endpoint returns records for a specified amount/date range after the code session check.

Forced error mode returns a 503 page for normal paths while information paths remain accessible. This is an intentional setting, not a proven database outage detector. The information login uses a short code and lacks a declared throttle/normal user authorization, so this interface requires hardening.

16.5 Activity and file storage

Activity records include user, action, description, model type/ID, properties, IP, user agent and date. The log screen supports filters. Logging calls are distributed through controllers/services; not every mutation is guaranteed to be covered.

PublicStorageSync creates/synchronizes storage links with a copy fallback. A settings helper can repair/sync files. Pre-order images use a public uploads directory. Purchase/expense/tax attachments can be public storage assets; protect access where documents are sensitive.

Sources: SettingController; PrivacyModeController; PrivacyModeService; DashboardVisibilityService; SuperAdminDashboardController; InformationController; SecretPos; LockdownIfDbError; ActivityLogController; PublicStorageSync.

17. Installation and configuration

17.1 Requirements derived from the project

The manifest requires PHP 8.2 or compatible later versions within ^8.2 and Laravel ^12.0. The frontend uses Vite ^7.0.7, Tailwind ^4 and Axios. PDF generation uses laravel-dompdf; spreadsheet import/export uses maatwebsite/excel. Install from composer.lock to preserve the reviewed dependency set. Node requirements should be checked against the locked/installed Vite package on the deployment machine.

Use a database server configured for Laravel. MySQL is the business deployment path reflected by repair/locking code; SQLite is used by the isolated tests. Required PHP capabilities include PDO for the chosen database, SQLite for tests, mbstring, XML/DOM, fileinfo, ZIP, OpenSSL and the extensions required by Composer's resolved packages.

An Apache/Nginx server must support the front-controller routes. For Apache, enable the needed rewrite configuration. The web document root should point to the project's public directory, not the repository root. Give the server write access to storage and bootstrap/cache without making the whole source tree broadly writable.

17.2 Safe fresh-install sequence

1. Copy the reviewed project and preserve composer.lock. Keep production data and configuration separate from source.
2. Install PHP dependencies using composer install. For production, use --no-dev --optimize-autoloader after resolving the deployment requirements.
3. Create .env from .env.example only on a fresh installation. Set application URL/environment, database settings, timezone and any mail/queue/cache/session configuration.
4. Generate APP_KEY once with php artisan key:generate for a new installation. Preserve the existing key during an upgrade/restore.
5. Create the empty database and run php artisan migrate. On a production release, back up first and run the approved migrations with --force through an authorized CLI deployment.
6. Create initial administrative accounts and reference data using a production-safe process. DatabaseSeeder currently adds demonstration records and resets some seeded users; do not run it blindly on live data.
7. Install frontend dependencies using npm install and run npm run build. Deploy generated public/build assets as required.
8. Run php artisan storage:link and verify logo/product/receipt/pre-order image access. Use the authenticated storage helper only when its fallback is needed.
9. Configure the business settings, stores, units, users, role permissions, products and opening balances in the order shown in Chapter 18.
10. Wire the real schedules into Laravel, verify php artisan schedule:list, then configure a once-per-minute scheduler invocation and any required queue worker.
11. Verify permission boundaries, sample purchases/sales/returns, printers, backup and restoration before handover.

17.3 Configuration groups

Group	What to supply
APP_*	URL, environment, key, debug flag and timezone
DB_*	Driver, database, host/port and restricted database user
SESSION/CACHE/QUEUE	Drivers appropriate to the server and working backing tables/services
MAIL_*	Mail transport and sender for tax invoice emails
Filesystem	Public storage link/copy configuration and writable upload areas
Database settings UI	Business identity, currency, printing, POS, tax versions and communication credentials

For production set APP_DEBUG=false and use secure transport/session configuration. Keep .env and provider credentials out of shared documentation and source control. No secret values are needed in a user manual.

The repository contains /migrate and /run-seed browser routes with no login requirement. They can modify the database and display diagnostics. Do not use them as the production installation procedure; remove or strongly restrict them and use controlled CLI deployment instead.

Sources: composer.json/lock; package.json; config/*; .env.example; bootstrap/app.php; public/.htaccess; setup.sh; DatabaseSeeder; routes/web.php.

18. What you need to do before handover

18.1 Business decisions to record

Decision	Required documented result
Pre-order stock	Choose reservation at save or deduction at completion; define zero-stock, linking, cancellation and reopening rules
Inventory ownership	Define one balance policy across products, price options and stores; select receipt vs order timing
Payment handling	Define credit, cash change, overpayments, advance use, deposits, refunds and cheque clearance
Accounting	Confirm accounts, opening balances, historical costing, bank tagging and posting/reversal rules
Tax setup	Owner/accountant-approved configured values, effective dates and official invoice data requirements
Permissions	Named roles, module actions, allowed stores, privileged control owners and export access
Operations	Backup owner, storage location, restore method, retention, schedule owner and support contacts

18.2 Initial data entry order

1. Business identity, timezone/currency and print settings.
2. Store records and default store.
3. Roles, named staff users and store assignments.
4. Category tree, brands, units and conversion factors.
5. Customers/suppliers and approved opening balances.
6. Chart accounts, banks, petty cash and owner opening equity.
7. Products, price options and verified opening stock by store.
8. Tax settings/version and product tax exceptions when required.
9. Notification configuration, reminder rules and scheduler validation.
10. Reconciliation of initial balances with signed-off source records.

18.3 Daily, weekly and monthly operations

Daily: check stock exceptions, due invoices and cheque state; record receipts/expenses promptly; reconcile tendered cash, cashbook, bank receipts and invoice payments; back up database/uploads. Do not use a masked or scaled view to establish the actual closing balance.

Weekly: compare product/price/store quantities, investigate negative balances, review outstanding supplier/customer items, inspect notification delivery results and failed jobs, and check that backups are readable.

Monthly: reconcile each bank, review category limits, verify tax records/payment balances, review owner movements, compare profit figures against historical cost evidence and test restore procedures on an isolated system according to the agreed schedule.

18.4 Handover document pack

This PDF provides the source-based feature/technical reference. Complete the handover with approved role/store access matrix, chosen business rules, production configuration record without secret values, illustrated user training, migration/data import record, backup/restore runbook, acceptance-test results, known-issue decisions and support contacts. Maintain a change log and update this manual when stock/payment rules change.

19. Review findings and prioritized action plan

Priority meanings: P0 = restrict before any public production exposure; P1 = resolve before relying on the affected stock/financial workflow; P2 = resolve during hardening and handover. Findings are confirmed source observations unless explicitly marked as requiring runtime verification. No application code was changed for this review.

R01 - P0: Public database-maintenance endpoints

The /migrate and /run-seed GET routes are outside the authenticated group. Migration performs schema repairs, including partial-column cleanup, and seeding writes demonstration/business records and resets selected seeded users. Error responses include recent log content. Restrict/remove web maintenance endpoints and use authorized CLI deployment. Acceptance: anonymous requests cannot run database changes or retrieve maintenance diagnostics.

Evidence: routes/web.php, final /migrate and /run-seed closures; DatabaseSeeder::run.

R02 - P0: Repository root may expose maintenance and sensitive files

The root .htaccess bypasses rewrites for real files/directories. If this directory is directly served, standalone PHP utilities such as clear_data.php can run and files such as CREDENTIALS.md may be retrievable. clear_data.php contains table truncation. Exact exposure depends on Apache configuration and was not tested. Serve public only; remove utility scripts from deployment and protect secrets/documents. Acceptance: external requests cannot execute root maintenance files or download non-public files.

Evidence: .htaccess; clear_data.php; seed_products.php; repository file layout.

R03 - P1: Pre-order stock and deposit contract is inconsistent

The suite has two failures and one error because save now deducts stock, rejects zero-stock linked products and reopening keeps stock deducted. Manual relinking deducts again; completing an unlinked item creates a zero-stock product without deduction. Pending edits reset paid/held totals, and completion does not fully transfer pending deposit/cheque linkage. New payment caps do not include all prior deposits. Define the desired policy, then implement consistent reservation/payment migration and regression checks.

Acceptance: create/edit/link/unlink/cancel/reopen/complete preserve stock exactly once; zero-stock behavior matches the approved policy; deposits/cheques survive completion with correct invoice, ledger and accounting references; all 21 existing pre-order tests are aligned and pass.

Evidence: PreOrderService::save/syncProduct/complete/reopen; PreOrderManagementTest tests 01, 02 and 13.

R04 - P1: Cheque lifecycle assumes every cheque belongs to a sale

ChequePaymentService::pass/return require a sale_id. Purchase own cheques, opening-balance cheques and pending pre-order cheques can exist without it. Automatic processing queries every eligible pending cheque, so those records can break processing. Bulk customer cheque payments also count as paid immediately, then pass may add a second payment. Implement separate supported source lifecycles and a single hold/clear policy.

Acceptance: each cheque type can hold/pass/return once; no duplicate cash movement; due and linked-order/purchase balances update; one unsupported record does not abort the automatic batch.

Evidence: ChequePaymentService; PurchaseController; BulkPaymentController; PreOrderService::addPayment.

R05 - P1: Stock balances differ across mutation paths

Purchases increase stock regardless of receipt status; shipment allocation only increases store stock; imports only set product stock. Returns, sale edits/deletes and purchase deletion do not consistently reverse every balance. Transfers discard source price identity and reversals can make destination stock negative. Use a shared stock movement service/journal with total, price, store and unit identity.

Acceptance: for each supported action, product quantity reconciles to approved branch/price balances; edits/deletions reverse original movements; repeated returns cannot exceed the unreturned quantity; concurrent sales cannot oversell.

Evidence: PurchaseController::store/destroy; InventoryStoreController::allocate/addStoreStock/updateTransfer/destroyTransfer; ProductController::import; SaleController; SaleReturnController; PurchaseReturnController.

R06 - P1: Stock write-offs and direct expenses skip normal postings

Write-off deducts product stock only and creates Expense without expense-accounting sync. POS seller card fees also create Expense directly. PettyCashExpense records are excluded from the ordinary Expense-based report. Use a consistent expense/stock posting path and eliminate duplicate or omitted expense reporting.

Acceptance: write-off, card fee and petty cash operations affect the correct inventory/account balances once and appear in the agreed expense/profit report.

Evidence: ProductWriteOffController::store; POSController::checkout; ExpenseAccountingService; AccountingController::storePettyExpense; ReportController::profitLoss.

R07 - P1: Scheduled functions are not registered

php artisan schedule:list returned "No scheduled tasks have been defined." Reminder/cheque declarations are in a legacy Console Kernel that is not connected by the current bootstrap. Register schedules in the active Laravel scheduling configuration, then configure cron and verify next-run output. Add repeat protection and last-day handling for reminders if required.

Acceptance: schedule:list lists the intended jobs; a controlled scheduler test executes one due job once with recorded outcome. Dashboard-triggered cheque mutation must follow the approved policy.

Evidence: bootstrap/app.php; routes/console.php; app/Console/Kernel.php; recorded schedule:list result.

R08 - P1: Granular authorization can be bypassed by alternate paths

POS checkout accepts edit_sale_id under pos.access without requiring sales.edit; return processing similarly runs through POS. Several authenticated read/export routes lack specific module permission. Store assignments are not consistently checked. Apply action-level permission and store/record checks at the controller/service boundary, including every AJAX path.

Acceptance: a cashier cannot edit/delete another sale or operate an unassigned store through direct requests; a role lacking the relevant read/export permission receives 403; approved administrator exceptions are explicit.

Evidence: routes/web.php; POSController::checkout; CustomerController::exportPdf; AccountingController::bankSystemBalance; User::hasStoreAccess.

R09 - P1: Public customer links lack scoped access tokens

Numeric customer/sale URLs expose bill/history/payment information without login. Ownership checks stop mismatched customer/sale pairs but do not prove the requester is the customer. Introduce signed, expiring or random scoped links with a business-approved expiry/revocation policy.

Acceptance: guessed IDs do not disclose customer records; valid issued links reveal only the intended customer/document and expire as configured.

Evidence: public customer routes; CustomerController::viewBill/viewHistory/paymentPage.

R10 - P1: Privileged code login needs stronger access control

The information interface can alter dashboard visibility and force normal pages into 503 mode using a short-code session. No normal user authorization or route throttle is declared. Replace or protect this mechanism with authenticated privileged access, rate limiting and a full audit trail. Add login throttling to the staff login as well.

Acceptance: unauthorized users cannot access or modify control settings; repeated failed authentication is limited; changes record the responsible user.

Evidence: InformationController::login/save; routes/web.php; LockdownIfDbError.

R11 - P1: Profit and legacy VAT reports are unsuitable for authoritative reconciliation

Profit-loss uses current product cost and does not explicitly filter sale_type= sale, so quotes can enter totals. Legacy VAT estimates current-rate tax from total amounts, rather than historical tax lines. Hidden/scaled data can affect reported figures. Snapshot historical cost, exclude quotations explicitly and designate the stored-tax reports for reconciliation; verify displayed totals are labelled when modified.

Acceptance: historical profits stay stable after product-cost changes; quotations do not affect realized profit; tax totals reconcile to stored lines and returns under an unrestricted view.

Evidence: ReportController::profitLoss/vat; DashboardController::index; DashboardVisibilityService; TaxManagementController.

R12 - P1: Payments and bank reconciliation need one consistent posting model

Purchase creation/edit changes paid totals without all corresponding Payment/accounting entries. Individual/bulk payments have inconsistent caps, cheque holds and transaction boundaries. Automated bank postings often omit a specific bank ID while bank balance fallback uses a shared account. Align document payments and accounting, tag every bank movement and make allocation atomic.

Acceptance: repeated requests cannot duplicate collections; payments cannot exceed approved remaining balance; all documents reconcile to their payment rows; each bank statement includes only that bank's movements.

Evidence: PurchaseController::store/update; PaymentController::store; BulkPaymentController; SalePaymentAccountingService; AccountingController::bankSystemBalanceAt.

R13 - P2: POS clear-cart route name is duplicated

POST pos/cart/clear and GET pos/cancel-edit both use pos.cart.clear. POS JavaScript posts to the generated named URL. The later registration can resolve it to the GET-only cancel-edit endpoint, causing method errors. Assign a unique cancel-edit name and verify all generated URLs.

Acceptance: cart clearing works in normal, quotation and edit modes and leaves no duplicate route names.

Evidence: routes/web.php POS cart routes; resources/views/pos/index.blade.php postJSON calls.

R14 - P2: Thirteen routes reference missing controller methods

Resource registrations advertise create/show/edit/update/delete beyond implemented actions in write-offs and returns. Five other actions reference absent views: users.show, brands.show, categories.show, units.create and units.show. Customer create/edit also reference absent standalone templates but use guarded fallback behavior. Restrict resource registration to supported actions or implement the missing workflows/templates. Appendix A marks missing actions and the five unguarded missing-view actions.

Acceptance: every advertised route has a working action; menus/forms expose only supported operations.

Evidence: route inventory compared to ProductWriteOffController, SaleReturnController and PurchaseReturnController methods; literal controller view references checked against resources/views.

R15 - P2: Fractional quantities and unit multipliers are inconsistent

Product stock is integer-based while store/price quantities allow decimals. Checkout clamps a multiplier to at least 1.0 and sale items do not consistently retain conversion identity for reversal. Establish base-unit decimal precision and persist the original multiplier.

Acceptance: pack, base-unit and fractional-unit sale/return/edit cycles restore exactly the original base stock.

Evidence: Product model; products migration; POSController::checkout; sale-item/reversal paths.

R16 - P2: Notification options and provider behavior need validation

Top5 is offered by the promotion form but rejected by validation. enable_sms_notifications is the actual service switch; provider-labelled endpoints need verification. WhatsApp counts represent links. Align form/controller options and report attempt/success/failure accurately. Verify stock flags are consumed by StockAlertService.

Acceptance: each offered audience/method works with controlled recipients; disabled sending stays disabled; errors are shown accurately and do not claim delivery from link generation.

Evidence: NotificationController::sendPromotion; notifications/send.blade.php; SmsService; WhatsAppService; StockAlertService.

R17 - P2: Rate fetches disable certificate verification

Exchange-rate HTTP requests use verify=false. Restore certificate verification and handle unavailable providers with a dated/stale indicator and approved manual fallback.

Acceptance: trusted HTTPS connections are verified; failures do not silently present an unverified current rate.

Evidence: ReportController::rates.

R18 - P2: Printed settings, attachments and recovery need acceptance checks

Not every prefix/paper/language flag is wired uniformly; purchase shipping expense creation is explicitly unfinished. Sensitive attachments may use public storage. Backups/restores and physical printing were not tested. Define supported output settings, restrict document access and rehearse database-plus-file restoration.

Acceptance: approved output sizes work; documented settings produce the expected result; private attachments require appropriate access; a full backup restores in isolation with matched stock/financial totals.

Evidence: SettingController; Sale::generateNumber; PurchaseController::store; TaxManagementController attachments; PublicStorageSync; file configuration.

20. Validation and acceptance-test plan

20.1 Checks performed

Check	Result
PHP syntax	221 source/config/migration/test files checked, no syntax errors
Laravel route inventory	325 routes; 121 route permission keys; 13 controller methods missing; 5 unguarded absent view templates
Named route consistency	Duplicate pos.cart.clear name found
Scheduler inspection	No tasks registered in the bootstrapped application
Existing PHPUnit suite	79 tests, 349 assertions; 76 pass, 2 failures, 1 error

Tests ran through the XAMPP PHP 8.2.4 executable with SQLite :memory:, array sessions/cache/mail and synchronous queues. No production migration or data seed was executed. --do-not-cache-result avoided rewriting the result cache.

The passing tests cover tax calculations and snapshots/returns/exports, invoice eligibility, expense accounting, bank reconciliation calculations, mixed sale cheque/payment accounting, customer name-only creation, privacy behavior, PDF letterhead coverage and most pre-order actions. Passing these cases does not prove coverage of every route or integration.

The failed tests were test_01_create_preorder_with_product_in_stock (9 instead of 10 stock), test_02_create_preorder_with_zero_stock_product_succeeds (insufficient-stock exception) and test_13_reopen_completed_order_reverses_sale_stock_payments_tax_and_accounting (4 instead of 5 stock).

20.2 Business acceptance scenarios

Scenario	Evidence to retain
Roles and branches	Cashier/manager/admin results for menu and direct action access; assigned/unassigned store checks
Purchase lifecycle	Pending/received, split payments, own/customer cheques, attachments, shipping and all stock balances
POS	Cash exact/change, split payment, credit, advance, selected branch/price/unit, tax bill and printing
Returns/exchanges	Partial/repeated/full returns, cash/account refund, mixed exchange and every stock/ledger balance
Pre-order	Zero stock, temporary item, link/relink/unlink, deposit, cheque, edit, cancel/reopen and completion
Accounting	Payment/expense/owner movements, petty cash, bank tagging, period trial balance and reversals
Tax	Inclusive/exclusive/zero/exempt, effective-date change, partial return, payment/adjustment and matched PDF/XLSX totals
Reports	Same screen/export filters; unrestricted versus masked views; quotation exclusion; historical cost stability
Integrations	Actual SMS test result, WhatsApp manual send, tax email attachment, failure reporting and scheduler logs
Operations	File access, print sizes, concurrent checkout, backup restore, deployment rollback and health response

Correct the P0/P1 findings first, then run the acceptance scenarios on a separate restored dataset. Record input, expected effect, actual effect, tester, date and sign-off. Retain unrestricted stock/balance snapshots before and after each transaction.

20.3 Scope boundaries

No dedicated repair-job/workshop management, full vehicle fleet registry, ecommerce storefront, online payment gateway, automatic WhatsApp API, QuickBooks connector or supplier ordering API was found in the reviewed routes/services. Vehicle details are part of pre-orders. Do not describe these unimplemented capabilities as existing system functions.

Appendix A. Complete registered endpoint and permission inventory

This appendix is generated from Laravel route:list for the reviewed working copy. GET includes HEAD. Normal business routes use auth and privacy_mode plus the listed permission. "Auth only" means no module-specific route gate; controller checks may still apply. "Public/code session" indicates an information endpoint that uses its own controller session check. "MISSING" means the controller action was not found; "VIEW MISSING" means the action references an absent unguarded template. Do not invoke maintenance endpoints simply to test this inventory.

Accounting / AccountingController

Source: app/Http/Controllers/Accounting/AccountingController.php

Method	Path	Action / Accounting Controller	Access gate
GET	/accounting	index	accounting.view
GET	/accounting/accounts	accounts	accounting.accounts
POST	/accounting/accounts	storeAccount	accounting.manage
GET	/accounting/balance-sheet	balanceSheet	accounting.balance-sheet
GET	/accounting/bank-book	bankBook	accounting.bank-book
GET	/accounting/banks	banks	accounting.banks
POST	/accounting/banks	storeBank	accounting.manage
POST	/accounting/banks/{bankAccount}/reconcile	reconcile	accounting.manage
GET	/accounting/banks/{bankAccount}/system-balance	bankSystemBalance	Auth only
GET	/accounting/cash-book	cashBook	accounting.cash-book
GET	/accounting/export/{section}/{format}	export	accounting.view
GET	/accounting/ledger	ledger	accounting.ledger
GET	/accounting/owner-equity	ownerEquity	accounting.owner-equity.view
POST	/accounting/owner-equity	storeOwnerEquity	accounting.owner-equity.create
PUT	/accounting/owner-equity/{ownerEquity}	updateOwnerEquity	accounting.owner-equity.edit
DELETE	/accounting/owner-equity/{ownerEquity}	destroyOwnerEquity	accounting.owner-equity.delete
GET	/accounting/petty-cash	pettyCash	accounting.petty-cash
POST	/accounting/petty-cash	storePettyFund	accounting.manage
POST	/accounting/petty-cash/expenses	storePettyExpense	accounting.manage
GET	/accounting/t-accounts	tAccounts	accounting.t-accounts
GET	/accounting/transactions	transactions	accounting.transactions
POST	/accounting/transactions	storeTransaction	accounting.manage
GET	/accounting/trial-balance	trialBalance	accounting.trial-balance

Accounting / PettyCashTransferController

Source: app/Http/Controllers/Accounting/PettyCashTransferController.php

Method	Path	Action / PettyCashTransferController	Access gate
GET	/accounting/petty-cash-transfer	create	accounting.manage
POST	/accounting/petty-cash-transfer	store	accounting.manage

ActivityLogController

Source: app/Http/Controllers/ActivityLogController.php

Method	Path	Action / ActivityLogController	Access gate
GET	/activity-log	index	activity-log.view

Auth / LoginController

Source: app/Http/Controllers/Auth/LoginController.php

Method	Path	Action / LoginController	Access gate
GET	/login	showLoginForm	Guest
POST	/login	login	Guest
POST	/logout	logout	Auth only

BrandController

Source: app/Http/Controllers/BrandController.php

Method	Path	Action / BrandController	Access gate
GET	/brands	index	brands.view
POST	/brands	store	brands.create
GET	/brands/create	create	brands.create
GET	/brands/{brand}	show [VIEW MISSING]	brands.view
PUT|PATCH	/brands/{brand}	update	brands.edit
DELETE	/brands/{brand}	destroy	brands.delete
GET	/brands/{brand}/edit	edit	brands.edit

BulkPaymentController

Source: app/Http/Controllers/BulkPaymentController.php

Method	Path	Action / BulkPaymentController	Access gate
GET	/customers/{customer}/bulk-payment	customerPayment	customers.edit
POST	/customers/{customer}/bulk-payment	storeCustomerPayment	customers.edit
GET	/suppliers/{supplier}/bulk-payment	supplierPayment	suppliers.edit
POST	/suppliers/{supplier}/bulk-payment	storeSupplierPayment	suppliers.edit

CategoryController

Source: app/Http/Controllers/CategoryController.php

Method	Path	Action / CategoryController	Access gate
GET	/categories	index	categories.view
POST	/categories	store	categories.create
GET	/categories/create	create	categories.create
GET	/categories/{category}	show [VIEW MISSING]	categories.view
PUT|PATCH	/categories/{category}	update	categories.edit
DELETE	/categories/{category}	destroy	categories.delete
GET	/categories/{category}/children	children	Auth only
GET	/categories/{category}/edit	edit	categories.edit

ChequePaymentController

Source: app/Http/Controllers/ChequePaymentController.php

Method	Path	Action / ChequePaymentController	Access gate
GET	/cheque-payments	index	cheque_payments.view
POST	/cheque-payments/{chequePayment}/pass	pass	cheque_payments.manage
POST	/cheque-payments/{chequePayment}/return	return	cheque_payments.manage

CustomerController

Source: app/Http/Controllers/CustomerController.php

Method	Path	Action / CustomerController	Access gate
GET	/customer/{customer}/bill/{sale}	viewBill	Public
GET	/customer/{customer}/history	viewHistory	Public
GET	/customer/{customer}/pay/{sale}	paymentPage	Public
GET	/customers	index	customers.view
POST	/customers	store	customers.create
GET	/customers/create	create	customers.create
GET	/customers/{customer}	show	customers.view
PUT|PATCH	/customers/{customer}	update	customers.edit
DELETE	/customers/{customer}	destroy	customers.delete
POST	/customers/{customer}/advance	storeAdvance	customers.edit
GET	/customers/{customer}/edit	edit	customers.edit
GET	/customers/{customer}/export-pdf/{type}	exportPdf	Auth only
POST	/customers/{customer}/send-reminder	sendPaymentReminder	customers.edit

DashboardController

Source: app/Http/Controllers/DashboardController.php

Method	Path	Action / Dashboard Controller	Access gate
GET	/dashboard	index	dashboard.view

ExpenseCategoryController

Source: app/Http/Controllers/ExpenseCategoryController.php

Method	Path	Action / ExpenseCategoryController	Access gate
GET	/expense-categories	index	expenses.view
POST	/expense-categories	store	expenses.create
GET	/expense-categories/create	create	expenses.create
PUT|PATCH	/expense-categories/{expense_category}	update	expenses.edit
DELETE	/expense-categories/{expense_category}	destroy	expenses.delete
GET	/expense-categories/{expense_category}/edit	edit	expenses.edit

ExpenseController

Source: app/Http/Controllers/ExpenseController.php

Method	Path	Action / ExpenseController	Access gate
GET	/expenses	index	expenses.view
POST	/expenses	store	expenses.create
GET	/expenses/create	create	expenses.create
GET	/expenses/{expense}	show	expenses.view
PUT|PATCH	/expenses/{expense}	update	expenses.edit
DELETE	/expenses/{expense}	destroy	expenses.delete
GET	/expenses/{expense}/edit	edit	expenses.edit

Framework and closure routes

Method	Path	Action / Framework and closure routes	Access gate
GET	/	Closure	Guest
GET	/migrate	Closure	Public
GET	/run-seed	Closure	Public
GET	/settings/storage-link	Closure	Auth only
GET	/storage/{path}	Closure	Public
GET	/up	Closure	Public

InformationController

Source: app/Http/Controllers/InformationController.php

Method	Path	Action / Information Controller	Access gate
GET	/information	card	Public
POST	/information/login	login	Public
GET	/information/range-bills	rangeBills	Public/code session
GET	/information/secret	secret	Public/code session
POST	/information/secret/save	save	Public/code session

InventoryStoreController

Source: app/Http/Controllers/InventoryStoreController.php

Method	Path	Action / InventoryStoreController	Access gate
GET	/stores	index	stores.view
GET	/stores/allocations	allocations	stores.allocations
GET	/stores/report	report	stores.report
GET	/stores/shipments	shipments	stores.shipments
POST	/stores/shipments	shipment	stores.manage
POST	/stores/shipments/allocate	allocate	stores.manage
GET	/stores/stores	stores	stores.stores
POST	/stores/stores	store	stores.manage
POST	/stores/stores/{store}/set-default	setAsDefault	stores.manage
GET	/stores/transfer-report	transferHistory	stores.transfer-report
GET	/stores/transfers	transfers	stores.transfers
POST	/stores/transfers	transfer	stores.manage
PUT	/stores/transfers/{transfer}	updateTransfer	stores.manage
DELETE	/stores/transfers/{transfer}	destroyTransfer	stores.manage

NotificationController

Source: app/Http/Controllers/NotificationController.php

Method	Path	Action / Notification Controller	Access gate
GET	/notifications	index	notifications.view
GET	/notifications/history	history	notifications.view
POST	/notifications/promotion/send	sendPromotion	notifications.configure
POST	/notifications/reminder/send-now	sendMonthlyRemindersNow	notifications.configure
GET	/notifications/send	sendForm	notifications.configure
GET	/notifications/settings	settings	notifications.configure

Method	Path	Action / Notification Controller	Access gate
POST	/notifications/settings/save	saveSettings	notifications.configure
POST	/notifications/{notification}/read	markRead	notifications.view

POSController

Source: app/Http/Controllers/POSController.php

Method	Path	Action / POSController	Access gate
GET	/api/customer-due/{id}	getCustomerDue	pos.access
GET	/api/sale-receipt/{id}	getSaleReceipt	pos.access
GET	/api/shop-details	getShopDetails	pos.access
GET	/pos	index	pos.access
GET	/pos/cancel-edit	cancelEdit	pos.access
POST	/pos/cart/add	addToCart	pos.access
POST	/pos/cart/add-return	addReturnItem	pos.access
POST	/pos/cart/clear	clearCart	pos.access
POST	/pos/cart/discount	setDiscount	pos.access
POST	/pos/cart/hold	holdCart	pos.access
GET	/pos/cart/holds	listHolds	pos.access
POST	/pos/cart/holds/load	loadHold	pos.access
POST	/pos/cart/holds/remove	removeHold	pos.access
POST	/pos/cart/item/update	updateCartItem	pos.access
POST	/pos/cart/remove	removeItem	pos.access
POST	/pos/cart/unit	setItemUnit	pos.access
POST	/pos/cart/update	updateQty	pos.access
POST	/pos/checkout	checkout	pos.access
POST	/pos/draft	saveDraft	pos.access
GET	/pos/products/{product}/prices	productPrices	pos.access
GET	/pos/recent-sales	recentSales	pos.access
GET	/pos/search-products	searchProducts	pos.access
POST	/pos/search-sale	searchSale	pos.access
GET	/quotations/create	quotationCreate	quotations.create

PaymentController

Source: app/Http/Controllers/PaymentController.php

Method	Path	Action / PaymentController	Access gate
POST	/payments	store	purchases.edit

Method	Path	Action / PaymentController	Access gate
GET	/payments/create	create	purchases.edit

PreOrderController

Source: app/Http/Controllers/PreOrderController.php

Method	Path	Action / PreOrderController	Access gate
GET	/preorders	index	preorder_view
POST	/preorders	store	preorder_create
GET	/preorders/create	create	preorder_create
GET	/preorders/customer-report	customerReport	preorder_reports
GET	/preorders/customer/{customer}/modal	customerModal	preorder_view
POST	/preorders/quick-customer	quickCustomer	preorder_create
GET	/preorders/report	report	preorder_reports
GET	/preorders/search-products	searchProducts	preorder_view
GET	/preorders/status/{status}	index	preorder_view
GET	/preorders/{preOrder}	show	preorder_view
PUT|PATCH	/preorders/{preOrder}	update	preorder_edit
POST	/preorders/{preOrder}/cancel	cancel	preorder_cancel
POST	/preorders/{preOrder}/complete	complete	preorder_complete
GET	/preorders/{preOrder}/edit	edit	preorder_edit
GET	/preorders/{preOrder}/invoice.pdf	invoicePdf	preorder_print_invoice
POST	/preorders/{preOrder}/items/{item}/price	changePrice	preorder_change_price
POST	/preorders/{preOrder}/items/{item}/sync	syncProduct	preorder_sync_product
POST	/preorders/{preOrder}/items/{item}/unsync	unsyncProduct	preorder_sync_product
POST	/preorders/{preOrder}/payments	addPayment	preorder_payment_create
DELETE	/preorders/{preOrder}/payments/{payment}	deletePayment	preorder_payment_edit
GET	/preorders/{preOrder}/quotation.pdf	quotationPdf	preorder_print_quotation
POST	/preorders/{preOrder}/reopen	reopen	preorder_reopen

PrivacyModeController

Source: app/Http/Controllers/PrivacyModeController.php

Method	Path	Action / PrivacyModeController	Access gate
POST	/privacy-mode/toggle	toggle	Auth only

ProductController

Source: app/Http/Controllers/ProductController.php

Method	Path	Action / ProductController	Access gate
GET	/products	index	products.view
POST	/products	store	products.create
GET	/products/barcode-print	barcodePrint	barcode.print
POST	/products/barcode-print/preview	barcodePrintPreview	barcode.print
GET	/products/barcode-search	barcodeSearch	barcode.print
GET	/products/create	create	products.create
GET	/products/import	importForm	products.create
POST	/products/import	import	products.create
GET	/products/import/template	downloadTemplate	products.create
GET	/products/{product}	show	products.view
PUT|PATCH	/products/{product}	update	products.edit
DELETE	/products/{product}	destroy	products.delete
GET	/products/{product}/edit	edit	products.edit
POST	/products/{product}/update-price	updatePrice	products.update-price

ProductPriceController

Source: app/Http/Controllers/ProductPriceController.php

Method	Path	Action / ProductPriceController	Access gate
POST	/product-prices	store	create_product_prices
PUT	/product-prices/{productPrice}	update	edit_product_prices
DELETE	/product-prices/{productPrice}	destroy	delete_product_prices
GET	/products/{product}/prices	index	view_product_prices

ProductWriteOffController

Source: app/Http/Controllers/ProductWriteOffController.php

Method	Path	Action / ProductWriteOffController	Access gate
GET	/product-write-offs	index	products.edit
POST	/product-write-offs	store	products.edit
GET	/product-write-offs/create	create [MISSING]	products.edit
GET	/product-write-offs/{product_write_off}	show [MISSING]	products.edit
PUT|PATCH	/product-write-offs/{product_write_off}	update [MISSING]	products.edit
DELETE	/product-write-offs/{product_write_off}	destroy [MISSING]	products.edit
GET	/product-write-offs/{product_write_off}/edit	edit [MISSING]	products.edit
GET	/products/write-off/index	index	products.edit

Method	Path	Action / ProductWriteOffController	Access gate
POST	/products/write-off/store	store	products.edit

PurchaseController

Source: app/Http/Controllers/PurchaseController.php

Method	Path	Action / PurchaseController	Access gate
GET	/purchases	index	purchases.view
POST	/purchases	store	purchases.create
GET	/purchases/create	create	purchases.create
GET	/purchases/{purchase}	show	purchases.view
PUT|PATCH	/purchases/{purchase}	update	purchases.edit
DELETE	/purchases/{purchase}	destroy	purchases.delete
GET	/purchases/{purchase}/edit	edit	purchases.edit

PurchaseReturnController

Source: app/Http/Controllers/PurchaseReturnController.php

Method	Path	Action / PurchaseReturnController	Access gate
GET	/purchase-returns	index	purchases.edit
POST	/purchase-returns	store	purchases.edit
GET	/purchase-returns/create	create	purchases.edit
GET	/purchase-returns/{purchase_return}	show [MISSING]	purchases.edit
PUT|PATCH	/purchase-returns/{purchase_return}	update [MISSING]	purchases.edit
DELETE	/purchase-returns/{purchase_return}	destroy [MISSING]	purchases.edit
GET	/purchase-returns/{purchase_return}/edit	edit [MISSING]	purchases.edit

ReportController

Source: app/Http/Controllers/ReportController.php

Method	Path	Action / ReportController	Access gate
GET	/reports/customer-due	customerDue	reports.customer-due
GET	/reports/customer-due/csv	customerDueCsv	reports.customer-due
GET	/reports/customer-due/pdf	customerDuePdf	reports.customer-due
GET	/reports/daily-ledger	dailyLedger	reports.sales
GET	/reports/daily-ledger/csv	dailyLedgerCsv	reports.sales
GET	/reports/daily-ledger/pdf	dailyLedgerPdf	reports.sales
GET	/reports/debit	debit	reports.debit

Method	Path	Action / ReportController	Access gate
GET	/reports/debit/csv	debitCsv	reports.debit
GET	/reports/debit/pdf	debitPdf	reports.debit
GET	/reports/due-bills	dueBills	reports.due-bills
GET	/reports/due-bills/csv	dueBillsCsv	reports.due-bills
GET	/reports/due-bills/pdf	dueBillsPdf	reports.due-bills
GET	/reports/expense	expense	reports.expense
GET	/reports/expense/csv	expenseCsv	reports.expense
GET	/reports/expense/pdf	expensePdf	reports.expense
GET	/reports/never-sold	neverSold	reports.never-sold
GET	/reports/profit-loss	profitLoss	reports.profit-loss
GET	/reports/profit-loss/csv	profitLossCsv	reports.profit-loss
GET	/reports/profit-loss/pdf	profitLossPdf	reports.profit-loss
GET	/reports/purchase	purchase	reports.purchase
GET	/reports/purchase/csv	purchaseCsv	reports.purchase
GET	/reports/purchase/pdf	purchasePdf	reports.purchase
GET	/reports/rates	rates	reports.rate-conversion
POST	/reports/rates/manual	saveManualRate	reports.rate-conversion
GET	/reports/receive	receive	reports.receive
GET	/reports/receive/csv	receiveCsv	reports.receive
GET	/reports/receive/pdf	receivePdf	reports.receive
GET	/reports/sales	sales	reports.sales
GET	/reports/sales/csv	salesCsv	reports.sales
GET	/reports/sales/pdf	salesPdf	reports.sales
GET	/reports/stock	stock	reports.stock
GET	/reports/stock/csv	stockCsv	reports.stock
GET	/reports/stock/pdf	stockPdf	reports.stock
GET	/reports/trending	trending	reports.trending
GET	/reports/unsold-recently	unsoldRecently	reports.unsold-recently
GET	/reports/vat	vat	reports.vat
GET	/reports/vat/csv	vatCsv	reports.vat
GET	/reports/vat/day-details	vatDayDetails	reports.vat
GET	/reports/vat/day/pdf	vatDayPdf	reports.vat
GET	/reports/vat/pdf	vatPdf	reports.vat

RoleController

Source: app/Http/Controllers/RoleController.php

Method	Path	Action / RoleController	Access gate
GET	/roles	index	roles.view
POST	/roles	store	roles.create
GET	/roles/create	create	roles.create
GET	/roles/{role}	show	roles.view
PUT|PATCH	/roles/{role}	update	roles.edit
DELETE	/roles/{role}	destroy	roles.delete
GET	/roles/{role}/edit	edit	roles.edit

SaleController

Source: app/Http/Controllers/SaleController.php

Method	Path	Action / SaleController	Access gate
GET	/quotations	quotations	quotations.view
POST	/quotations/{sale}/convert	convertQuotation	quotations.edit
GET	/quotations/{sale}/pdf	quotationPdf	quotations.view
GET	/sales	index	sales.view
POST	/sales	store	sales.create
GET	/sales-export/csv	exportCsv	sales.view
GET	/sales-export/pdf	exportPdf	sales.view
GET	/sales/create	create	sales.create
POST	/sales/payment	addPayment	sales.edit
GET	/sales/{sale}	show	sales.view
PUT|PATCH	/sales/{sale}	update	sales.edit
DELETE	/sales/{sale}	destroy	sales.delete
GET	/sales/{sale}/edit	edit	sales.edit
GET	/sales/{sale}/print	print	sales.view
POST	/sales/{sale}/return	returnSale	sales.edit

SaleReturnController

Source: app/Http/Controllers/SaleReturnController.php

Method	Path	Action / SaleReturnController	Access gate
GET	/sale-returns	index	sales.edit
POST	/sale-returns	store	sales.edit
GET	/sale-returns/create	create	sales.edit
GET	/sale-returns/{id}/print	print	sales.edit

Method	Path	Action / SaleReturnController	Access gate
GET	/sale-returns/{sale_return}	show [MISSING]	sales.edit
PUT|PATCH	/sale-returns/{sale_return}	update [MISSING]	sales.edit
DELETE	/sale-returns/{sale_return}	destroy [MISSING]	sales.edit
GET	/sale-returns/{sale_return}/edit	edit [MISSING]	sales.edit

SettingController

Source: app/Http/Controllers/SettingController.php

Method	Path	Action / SettingController	Access gate
GET	/settings	index	settings.view
GET	/settings/barcode	barcode	settings.view
GET	/settings/business	business	settings.view
GET	/settings/general	general	settings.view
GET	/settings/invoice	invoice	settings.view
GET	/settings/pos	pos	settings.view
GET	/settings/preorder	preorder	settings.view
GET	/settings/quotation	quotation	settings.view
POST	/settings/save	save	settings.edit

SuperAdminDashboardController

Source: app/Http/Controllers/SuperAdminDashboardController.php

Method	Path	Action / SuperAdminDashboardController	Access gate
GET	/fun/dashboard	index	Super Admin
POST	/fun/dashboard/save	save	Super Admin
GET	/fun/dashboard/{section}	index	Super Admin
GET	/fun/search/customers	searchCustomers	Super Admin
GET	/fun/search/products	searchProducts	Super Admin
GET	/fun/search/sales	searchSales	Super Admin
GET	/fun/search/suppliers	searchSuppliers	Super Admin

SupplierController

Source: app/Http/Controllers/SupplierController.php

Method	Path	Action / SupplierController	Access gate
GET	/suppliers	index	suppliers.view
POST	/suppliers	store	suppliers.create

Method	Path	Action / SupplierController	Access gate
GET	/suppliers/create	create	suppliers.create
GET	/suppliers/{supplier}	show	suppliers.view
PUT|PATCH	/suppliers/{supplier}	update	suppliers.edit
DELETE	/suppliers/{supplier}	destroy	suppliers.delete
GET	/suppliers/{supplier}/edit	edit	suppliers.edit

TaxManagementController

Source: app/Http/Controllers/TaxManagementController.php

Method	Path	Action / TaxManagementController	Access gate
GET	/sales/{sale}/tax-invoice	taxInvoice	tax.invoice.print
POST	/sales/{sale}/tax-invoice/email	emailTaxInvoice	tax.invoice.print
GET	/tax	dashboard	tax.view
GET	/tax/adjustments	adjustments	tax.view
POST	/tax/adjustments	storeAdjustment	tax.adjustment.manage
POST	/tax/adjustments/{taxAdjustment}/approve	approveAdjustment	tax.adjustment.manage
GET	/tax/dashboard/details/{metric}	dashboardDetails	tax.view
GET	/tax/ledger/{direction}	ledger	tax.view
GET	/tax/payments	payments	tax.view
POST	/tax/payments	storePayment	tax.payment.create
GET	/tax/payments/balance	paymentBalance	tax.view
PATCH	/tax/payments/{taxPayment}	updatePayment	tax.payment.edit
DELETE	/tax/payments/{taxPayment}	destroyPayment	tax.payment.delete
POST	/tax/payments/{taxPayment}/finalize	finalizePayment	tax.payment.edit
POST	/tax/payments/{taxPayment}/reverse	reversePayment	tax.payment.edit
GET	/tax/reports	reports	tax.reports.view
GET	/tax/reports/export/{format}	exportReport	tax.reports.export

UnitController

Source: app/Http/Controllers/UnitController.php

Method	Path	Action / UnitController	Access gate
GET	/units	index	units.view
POST	/units	store	units.create
GET	/units/create	create [VIEW MISSING]	units.create
GET	/units/{unit}	show [VIEW MISSING]	units.view

Method	Path	Action / UnitController	Access gate
PUT|PATCH	/units/{unit}	update	units.edit
DELETE	/units/{unit}	destroy	units.delete
GET	/units/{unit}/edit	edit	units.edit

UserController

Source: app/Http/Controllers/UserController.php

Method	Path	Action / UserController	Access gate
GET	/users	index	users.view
POST	/users	store	users.create
GET	/users/create	create	users.create
GET	/users/{user}	show [VIEW MISSING]	users.view
PUT|PATCH	/users/{user}	update	users.edit
DELETE	/users/{user}	destroy	users.delete
GET	/users/{user}/edit	edit	users.edit

Appendix B. Data model and relationship reference

This appendix lists all 51 application model files, their declared mass-assignable fields and their explicit Eloquent relationships. It is a source-level data dictionary, not a dump of business records or a guarantee of the deployed database schema. Created/updated timestamps, inherited fields and derived attributes may not be in the fillable list. The 67 migrations are the authoritative schema-change sources; many-to-many/framework tables may have no standalone model.

Accounting/AccountTransaction

Model reference	Declared structure / Accounting/AccountTransaction
Fields	account_id, related_account_id, bank_account_id, user_id, transaction_date, direction, payment_method, amount, cheque_number, reference_no, source_type, source_id, description, is_reconciled, reconciled_at
Relationships	account -> belongsTo(ChartAccount::class, 'account_id') relatedAccount -> belongsTo(ChartAccount::class, 'related_account_id') bankAccount -> belongsTo(BankAccount::class) user -> belongsTo(User::class)
Casts	transaction_date: date, amount: decimal:2, is_reconciled: boolean, reconciled_at: datetime

Accounting/BankAccount

Model reference	Declared structure / Accounting/BankAccount
Fields	chart_account_id, bank_name, account_name, account_number, opening_balance, statement_balance, is_active
Relationships	chartAccount -> belongsTo(ChartAccount::class, 'chart_account_id') reconciliations -> hasMany(BankReconciliation::class)
Casts	opening_balance: decimal:2, statement_balance: decimal:2, is_active: boolean

Accounting/BankReconciliation

Model reference	Declared structure / Accounting/BankReconciliation
Fields	bank_account_id, user_id, statement_date, statement_balance, system_balance, difference, status, notes
Relationships	bankAccount -> belongsTo(BankAccount::class) user -> belongsTo(User::class)
Casts	statement_date: date, statement_balance: decimal:2, system_balance: decimal:2, difference: decimal:2

Accounting/ChartAccount

Model reference	Declared structure / Accounting/ChartAccount
Fields	code, name, type, subtype, opening_balance, current_balance, is_system, is_active
Relationships	transactions -> hasMany(AccountTransaction::class, 'account_id') bankAccount -> hasOne(BankAccount::class, 'chart_account_id')
Casts	opening_balance: decimal:2, current_balance: decimal:2, is_system: boolean, is_active: boolean

Accounting/OwnerEquityMovement

Model reference	Declared structure / Accounting/OwnerEquityMovement
Fields	type, owner_name, movement_date, payment_account, amount, reference_no, note, asset_account_id, equity_account_id, asset_transaction_id, equity_transaction_id, user_id
Relationships	assetAccount -> belongsTo(ChartAccount::class, 'asset_account_id') equityAccount -> belongsTo(ChartAccount::class, 'equity_account_id') assetTransaction -> belongsTo(AccountTransaction::class, 'asset_transaction_id') equityTransaction -> belongsTo(AccountTransaction::class, 'equity_transaction_id') user -> belongsTo(User::class)
Casts	movement_date: date, amount: decimal:2

Accounting/PettyCashExpense

Model reference	Declared structure / Accounting/PettyCashExpense
Fields	petty_cash_fund_id, expense_category_id, user_id, expense_date, amount, voucher_no, description
Relationships	fund -> belongsTo(PettyCashFund::class, 'petty_cash_fund_id') category -> belongsTo(ExpenseCategory::class, 'expense_category_id') user -> belongsTo(User::class)
Casts	expense_date: date, amount: decimal:2

Accounting/PettyCashFund

Model reference	Declared structure / Accounting/PettyCashFund
Fields	chart_account_id, name, opening_balance, current_balance, is_active
Relationships	chartAccount -> belongsTo(ChartAccount::class, 'chart_account_id') expenses -> hasMany(PettyCashExpense::class)
Casts	opening_balance: decimal:2, current_balance: decimal:2, is_active: boolean

ActivityLog

Model reference	Declared structure / ActivityLog
Fields	user_id, action, model_type, model_id, description, changes, ip_address
Relationships	user -> belongsTo(User::class)
Casts	changes: array

Brand

Model reference	Declared structure / Brand
Fields	name, description, logo, is_active
Relationships	products -> belongsToMany(Product::class, 'brand_product')
Casts	is_active: boolean

Category

Model reference	Declared structure / Category
Fields	name, description, parent_id, is_active
Relationships	parent -> belongsTo(Category::class, 'parent_id') children -> hasMany(Category::class, 'parent_id') products -> belongsToMany(Product::class, 'category_product')
Casts	is_active: boolean

ChequePayment

Model reference	Declared structure / ChequePayment
Fields	type, sale_id, purchase_id, supplier_id, pre_order_id, customer_id, payment_id, user_id, processed_by, cheque_date, cheque_number, bank_name, account_name, amount, status, processed_at, auto_passed, notes
Relationships	sale -> belongsTo(Sale::class) purchase -> belongsTo(Purchase::class) preOrder -> belongsTo(PreOrder::class) customer -> belongsTo(Customer::class) supplier -> belongsTo(Supplier::class) payment -> belongsTo(Payment::class) user -> belongsTo(User::class) processor -> belongsTo(User::class, 'processed_by')
Casts	cheque_date: date, amount: decimal:2, processed_at: datetime, auto_passed: boolean

Customer

Model reference	Declared structure / Customer
Fields	name, tin, email, phone, address, city, country, opening_balance, is_active
Relationships	sales -> hasMany(Sale::class) payments -> hasMany(Payment::class) preOrders -> hasMany(PreOrder::class)
Casts	opening_balance: decimal:2, is_active: boolean
Other behavior	getDueAmountAttribute, getSalesDueAmountAttribute, getAdvanceBalanceAttribute, getPreOrderDueAmountAttribute

Expense

Model reference	Declared structure / Expense
Fields	expense_category_id, user_id, expense_date, amount, payment_method, description, receipt, store_id
Relationships	store -> belongsTo(Store::class) category -> belongsTo(ExpenseCategory::class, 'expense_category_id') user -> belongsTo(User::class)
Casts	expense_date: date, amount: decimal:2

ExpenseCategory

Model reference	Declared structure / ExpenseCategory
Fields	name, description, is_active, limit, reset_frequency, reset_date
Relationships	expenses -> hasMany(Expense::class)
Casts	is_active: boolean, limit: decimal:2, reset_date: integer

Payment

Model reference	Declared structure / Payment
Fields	purchase_id, supplier_id, sale_id, pre_order_id, customer_id, user_id, amount, payment_method, reference_no, payment_date, notes, store_id
Relationships	store -> belongsTo(Store::class) purchase -> belongsTo(Purchase::class) supplier -> belongsTo(Supplier::class) sale -> belongsTo(Sale::class) preOrder -> belongsTo(PreOrder::class) customer -> belongsTo(Customer::class) user -> belongsTo(User::class)
Casts	amount: decimal:2, payment_date: date

PreOrder

Model reference	Declared structure / PreOrder
Fields	pre_order_number, customer_id, store_id, sale_id, pre_order_date, document_type, vehicle_name, registration_number, chassis_number, vehicle_description, vehicle_image, instructions, notes, expected_delivery_date, bill_discount_type, bill_discount_value, custom_tax_rate, pdf_tax_display, subtotal, discount_amount, tax_amount, rounding_adjustment, grand_total, paid_amount, held_cheque_amount, due_amount, status, payment_status, completed_at, completed_by, cancelled_at, cancelled_by, cancellation_reason, created_by, updated_by
Relationships	customer -> belongsTo(Customer::class) store -> belongsTo(Store::class) sale -> belongsTo(Sale::class) items -> hasMany(PreOrderItem::class) payments -> hasMany(Payment::class) chequePayments -> hasMany(ChequePayment::class) activities -> hasMany(PreOrderActivity::class) creator -> belongsTo(User::class, 'created_by') updater -> belongsTo(User::class, 'updated_by') completer -> belongsTo(User::class, 'completed_by') canceller -> belongsTo(User::class, 'cancelled_by')
Casts	pre_order_date: date, expected_delivery_date: date, completed_at: datetime, cancelled_at: datetime, bill_discount_value: decimal:2, subtotal: decimal:2, discount_amount: decimal:2, tax_amount: decimal:2, rounding_adjustment: decimal:4, grand_total: decimal:2, paid_amount: decimal:2, held_cheque_amount: decimal:2, due_amount: decimal:2
Other behavior	generateNumber, getVehicleImageUrlAttribute, getVehicleImageAbsolutePathAttribute

PreOrderActivity

Model reference	Declared structure / PreOrderActivity
Fields	pre_order_id, user_id, action, description, old_values, new_values, ip_address
Relationships	preOrder -> belongsTo(PreOrder::class) user -> belongsTo(User::class)
Casts	old_values: array, new_values: array

PreOrderItem

Model reference	Declared structure / PreOrderItem
Fields	pre_order_id, product_id, product_price_id, original_product_name, description, quantity, quoted_price, final_price, discount_type, discount_value, gross_amount, discount_amount, tax_amount, line_total, sync_status, tax_snapshot, notes
Relationships	preOrder -> belongsTo(PreOrder::class) product -> belongsTo(Product::class) productPrice -> belongsTo(ProductPrice::class)
Casts	quantity: integer, quoted_price: decimal:2, final_price: decimal:2, discount_value: decimal:2, gross_amount: decimal:2, discount_amount: decimal:2, tax_amount: decimal:2, line_total: decimal:2, tax_snapshot: array

PrivacyModeLog

Model reference	Declared structure / PrivacyModeLog
Fields	user_id, action, page, ip_address, created_at
Relationships	user -> belongsTo(User::class)

PrivacyModeSetting

Model reference	Declared structure / PrivacyModeSetting
Fields	is_enabled, shortcut_key, shortcut_key_mac, visible_invoice_limit, sales_list_percentage_mode, masking_type, apply_to_pos, apply_to_sales_list, apply_to_reports, apply_to_dashboard, apply_to_customer_history, created_by, updated_by
Relationships	creator -> belongsTo(User::class, 'created_by') updater -> belongsTo(User::class, 'updated_by')
Casts	is_enabled: boolean, apply_to_pos: boolean, apply_to_sales_list: boolean, apply_to_reports: boolean, apply_to_dashboard: boolean, apply_to_customer_history: boolean, visible_invoice_limit: integer

Product

Model reference	Declared structure / Product
Fields	is_pre_order, name, sku, category_id, brand_id, unit_id, visible_units, description, cost_price, selling_price, stock_quantity, alert_quantity, image, barcode, is_active
Relationships	category -> belongsTo(Category::class) categories -> belongsToMany(Category::class, 'category_product') brand -> belongsTo(Brand::class) brands -> belongsToMany(Brand::class, 'brand_product') unit -> belongsTo(Unit::class) purchaseItems -> hasMany(PurchaseItem::class) prices -> hasMany(ProductPrice::class) activePrices -> hasMany(ProductPrice::class) defaultPrice -> hasOne(ProductPrice::class) saleItems -> hasMany(SaleItem::class) preOrderItems -> hasMany(PreOrderItem::class) storeStocks -> hasMany(StoreStock::class) excludedStores -> belongsToMany(Store::class, 'product_store_exclusions') taxSetting -> hasOne(ProductTaxSetting::class)
Casts	cost_price: decimal:2, selling_price: decimal:2, stock_quantity: integer, alert_quantity: integer, is_active: boolean, base_unit: string, unit_factors: array, visible_units: array
Other behavior	unitFactor, priceForUnit, basePriceFromUnit, isLowStock

ProductPrice

Model reference	Declared structure / ProductPrice
Fields	product_id, cost_price, selling_price, stock_qty, is_default, status
Relationships	product -> belongsTo(Product::class) purchaseItems -> hasMany(PurchaseItem::class) saleItems -> hasMany(SaleItem::class)
Casts	cost_price: decimal:2, selling_price: decimal:2, stock_qty: decimal:3, is_default: boolean

ProductTaxSetting

Model reference	Declared structure / ProductTaxSetting
Fields	product_id, tax_type_id, tax_status, vat_rate, sale_price_mode, purchase_price_mode, output_vat_allowed, input_vat_allowed
Relationships	product -> belongsTo(Product::class) taxType -> belongsTo(TaxType::class)
Casts	vat_rate: decimal:4, output_vat_allowed: boolean, input_vat_allowed: boolean

Purchase

Model reference	Declared structure / Purchase
Fields	is_pre_order, purchase_no, reference_no, supplier_tax_invoice_number, supplier_tax_invoice_date, purchase_vat_mode, supplier_id, store_id, stock_shipment_id, user_id, purchase_date, status, discount_type, discount_amount, tax_id, tax_amount, taxable_purchase_value, input_vat_claimable, tax_period, tax_snapshot, shipping_cost, shipping_type, payment_method, cheque_number, bank_reference, total_amount, paid_amount, held_own_cheque_amount, due_amount, payment_status, document_path, notes
Relationships	supplier -> belongsTo(Supplier::class) store -> belongsTo(\App\Models\Store::class) user -> belongsTo(User::class) items -> hasMany(PurchaseItem::class) payments -> hasMany(Payment::class) taxLines -> hasMany(TransactionTaxLine::class, 'transaction_id')
Casts	purchase_date: date, total_amount: decimal:2, paid_amount: decimal:2, held_own_cheque_amount: decimal:2, due_amount: decimal:2, discount_amount: decimal:2, tax_amount: decimal:2, taxable_purchase_value: decimal:4, input_vat_claimable: boolean, tax_snapshot: array, supplier_tax_invoice_date: date, shipping_cost: decimal:2

PurchaseItem

Model reference	Declared structure / PurchaseItem
Fields	purchase_id, product_id, product_price_id, quantity, unit_cost, selling_price, total
Relationships	purchase -> belongsTo(Purchase::class) product -> belongsTo(Product::class) productPrice -> belongsTo(ProductPrice::class)
Casts	quantity: integer, unit_cost: decimal:2, selling_price: decimal:2, total: decimal:2

PurchaseReturn

Model reference	Declared structure / PurchaseReturn
Fields	purchase_id, user_id, return_date, total_refund, notes
Relationships	purchase -> belongsTo(Purchase::class) user -> belongsTo(User::class) items -> hasMany(PurchaseReturnItem::class)
Casts	return_date: date, total_refund: decimal:2

PurchaseReturnItem

Model reference	Declared structure / PurchaseReturnItem
Fields	purchase_return_id, purchase_item_id, product_id, quantity, unit_price, total
Relationships	return -> belongsTo(PurchaseReturn::class, 'purchase_return_id') product -> belongsTo(Product::class) purchaseItem -> belongsTo(PurchaseItem::class)
Casts	quantity: integer, unit_price: decimal:2, total: decimal:2

Role

Model reference	Declared structure / Role
Fields	name, description, permissions, is_active
Relationships	users -> hasMany(User::class)
Casts	permissions: array, is_active: boolean
Other behavior	hasPermission

Sale

Model reference	Declared structure / Sale
Fields	sale_no, tax_invoice_number, tax_template_version, customer_id, user_id, store_id, sale_date, subtotal, tax, rounding_adjustment, tax_snapshot, discount, total_amount, paid_amount, held_cheque_amount, tendered_amount, due_amount, payment_status, payment_method, cheque_number, bank_reference, sale_type, notes
Relationships	customer -> belongsTo(Customer::class) user -> belongsTo(User::class) items -> hasMany(SaleItem::class) payments -> hasMany(Payment::class) chequePayments -> hasMany(ChequePayment::class) preOrder -> hasOne(PreOrder::class) returns -> hasMany(SaleReturn::class) exchangeReturns -> hasMany(SaleReturn::class, 'exchange_sale_id') store -> belongsTo(Store::class) taxLines -> hasMany(TransactionTaxLine::class, 'transaction_id')
Casts	sale_date: date, subtotal: decimal:2, tax: decimal:2, rounding_adjustment: decimal:4, tax_snapshot: array, discount: decimal:2, total_amount: decimal:2, paid_amount: decimal:2, held_cheque_amount: decimal:2, tendered_amount: decimal:2, due_amount: decimal:2
Other behavior	generateNumber

SaleItem

Model reference	Declared structure / SaleItem
Fields	sale_id, product_id, product_price_id, quantity, unit_price, total
Relationships	sale -> belongsTo(Sale::class) product -> belongsTo(Product::class) productPrice -> belongsTo(ProductPrice::class)
Casts	quantity: integer, unit_price: decimal:2, total: decimal:2

SaleReturn

Model reference	Declared structure / SaleReturn
Fields	sale_id, exchange_sale_id, user_id, return_date, subtotal, total_refund, notes
Relationships	sale -> belongsTo(Sale::class) exchangeSale -> belongsTo(Sale::class, 'exchange_sale_id') user -> belongsTo(User::class) items -> hasMany(SaleReturnItem::class)
Casts	return_date: datetime, subtotal: float, total_refund: float

SaleReturnItem

Model reference	Declared structure / SaleReturnItem
Fields	sale_return_id, sale_item_id, product_id, product_price_id, quantity, unit_price, total
Relationships	return -> belongsTo(SaleReturn::class, 'sale_return_id') product -> belongsTo(Product::class) saleItem -> belongsTo(SaleItem::class) productPrice -> belongsTo(ProductPrice::class)
Casts	quantity: integer, unit_price: decimal:2, total: decimal:2

SecretSetting

Model reference	Declared structure / SecretSetting
Fields	key, value, type, group

Setting

Model reference	Declared structure / Setting
Fields	key, value, type, group

StockShipment

Model reference	Declared structure / StockShipment
Fields	shipment_no, grn_no, supplier_id, shipment_date, received_date, status, freight_cost, duty_cost, other_cost, notes
Relationships	supplier -> belongsTo(Supplier::class) items -> hasMany(StockShipmentItem::class)
Casts	shipment_date: date, received_date: date, freight_cost: decimal:2, duty_cost: decimal:2, other_cost: decimal:2

StockShipmentAllocation

Model reference	Declared structure / StockShipmentAllocation
Fields	stock_shipment_item_id, store_id, quantity
Casts	quantity: decimal:3

StockShipmentItem

Model reference	Declared structure / StockShipmentItem
Fields	stock_shipment_id, product_id, quantity, unit_cost, landed_unit_cost, selling_price
Relationships	shipment -> belongsTo(StockShipment::class, 'stock_shipment_id') product -> belongsTo(Product::class) allocations -> hasMany(StockShipmentAllocation::class)
Casts	quantity: decimal:3, unit_cost: decimal:2, landed_unit_cost: decimal:2, selling_price: decimal:2

Store

Model reference	Declared structure / Store
Fields	name, code, phone, address, is_default, is_active
Relationships	stocks -> hasMany(StoreStock::class)
Casts	is_default: boolean, is_active: boolean

StoreStock

Model reference	Declared structure / StoreStock
Fields	store_id, product_id, product_price_id, quantity
Relationships	store -> belongsTo(Store::class) product -> belongsTo(Product::class)
Casts	quantity: decimal:3

StoreStockTransfer

Model reference	Declared structure / StoreStockTransfer
Fields	store_transfer_id, from_store_id, to_store_id, product_id, quantity, transfer_date, reference_no, notes, shipping_cost, additional_expense
Relationships	transfer -> belongsTo(StoreTransfer::class, 'store_transfer_id') product -> belongsTo(Product::class) fromStore -> belongsTo(Store::class, 'from_store_id') toStore -> belongsTo(Store::class, 'to_store_id')
Casts	quantity: decimal:3, transfer_date: date, shipping_cost: decimal:2, additional_expense: decimal:2

StoreTransfer

Model reference	Declared structure / StoreTransfer
Fields	transfer_no, from_store_id, to_store_id, transfer_date, reference_no, notes, shipping_cost, additional_expense
Relationships	fromStore -> belongsTo(Store::class, 'from_store_id') toStore -> belongsTo(Store::class, 'to_store_id') items -> hasMany(StoreStockTransfer::class, 'store_transfer_id')
Casts	transfer_date: date, shipping_cost: decimal:2, additional_expense: decimal:2

Supplier

Model reference	Declared structure / Supplier
Fields	name, tin, company_name, email, phone, address, city, country, opening_balance, is_active
Relationships	purchases -> hasMany(Purchase::class) payments -> hasMany(Payment::class)
Casts	opening_balance: decimal:2, is_active: boolean
Other behavior	getDueAmountAttribute

SystemNotification

Model reference	Declared structure / SystemNotification
Fields	user_id, type, title, message, data, read_at
Relationships	user -> belongsTo(User::class)
Casts	data: array, read_at: datetime

TaxAdjustment

Model reference	Declared structure / TaxAdjustment
Fields	No explicit fillable list
Casts	adjustment_date: date, amount: decimal:4, approved_at: datetime

TaxLedgerEntry

Model reference	Declared structure / TaxLedgerEntry
Fields	No explicit fillable list
Casts	entry_date: date, taxable_amount: decimal:4, tax_amount: decimal:4, adjustment_amount: decimal:4

TaxPayment

Model reference	Declared structure / TaxPayment
Fields	No explicit fillable list
Casts	payment_date: date, output_vat: decimal:4, input_vat: decimal:4, adjustments: decimal:4, previous_balance: decimal:4, payable_amount: decimal:4, paid_amount: decimal:4, remaining_amount: decimal:4, finalized_at: datetime

TaxSetting

Model reference	Declared structure / TaxSetting
Fields	version, vat_enabled, vat_registered, supplier_tin, default_vat_rate, default_sale_price_mode, default_purchase_price_mode, customer_invoice_vat_display, regular_invoice_vat_note, allow_product_override, invoice_prefix, starting_invoice_number, next_invoice_number, branch_code, active_template_version, effective_from, effective_to, created_by
Casts	vat_enabled: boolean, vat_registered: boolean, default_vat_rate: decimal:4, regular_invoice_vat_note: boolean, allow_product_override: boolean, effective_from: date, effective_to: date

TaxType

Model reference	Declared structure / TaxType
Fields	name, code, rate, is_enabled, effective_from, effective_to, created_by
Casts	rate: decimal:4, is_enabled: boolean, effective_from: date, effective_to: date

TransactionTaxLine

Model reference	Declared structure / TransactionTaxLine
Fields	No explicit fillable list
Casts	original_unit_price: decimal:4, quantity: decimal:4, gross_amount: decimal:4, discount_amount: decimal:4, taxable_amount: decimal:4, tax_rate: decimal:4, vat_amount: decimal:4, total_amount: decimal:4, rounding_adjustment: decimal:4, input_vat_claimable: boolean, setting_snapshot: array

Unit

Model reference	Declared structure / Unit
Fields	name, short_name, base_unit_multiplier, is_active
Relationships	products -> hasMany(Product::class)
Casts	base_unit_multiplier: decimal:3, is_active: boolean

User

Model reference	Declared structure / User
Fields	name, username, email, password, phone, role_id, is_active
Relationships	role -> belongsTo(Role::class) stores -> belongsToMany(Store::class) sales -> hasMany(Sale::class) purchases -> hasMany(Purchase::class) expenses -> hasMany(Expense::class) preOrders -> hasMany(PreOrder::class, 'created_by')
Other behavior	hasStoreAccess, hasPermission, isSystemAdministrator, isSuperAdmin

Migration-created table inventory

Includes framework and pivot tables. Later migrations add/change columns. This listing is derived from Schema::create calls, not the live database.

Table	Creation migration
users	0001_01_01_000000_create_users_table.php
password_reset_tokens	0001_01_01_000000_create_users_table.php
sessions	0001_01_01_000000_create_users_table.php
cache	0001_01_01_000001_create_cache_table.php
cache_locks	0001_01_01_000001_create_cache_table.php
jobs	0001_01_01_000002_create_jobs_table.php
job_batches	0001_01_01_000002_create_jobs_table.php
failed_jobs	0001_01_01_000002_create_jobs_table.php
roles	2024_01_01_000003_create_roles_table.php
suppliers	2024_01_01_000005_create_suppliers_table.php
categories	2024_01_01_000006_create_categories_table.php
brands	2024_01_01_000007_create_brands_table.php
units	2024_01_01_000008_create_units_table.php

Table	Creation migration
products	2024_01_01_000009_create_products_table.php
purchases	2024_01_01_000010_create_purchases_table.php
purchase_items	2024_01_01_000010_create_purchases_table.php
customers	2024_01_01_000011_create_customers_table.php
sales	2024_01_01_000012_create_sales_table.php
sale_items	2024_01_01_000012_create_sales_table.php
expense_categories	2024_01_01_000013_create_expenses_table.php
expenses	2024_01_01_000013_create_expenses_table.php
settings	2024_01_01_000014_create_settings_table.php
activity_logs	2024_01_01_000015_create_activity_logs_table.php
payments	2025_11_10_000000_create_payments_table.php
sale_returns	2025_11_12_000100_create_sale_returns_table.php
sale_return_items	2025_11_12_000101_create_sale_return_items_table.php
system_notifications	2025_11_24_120000_create_system_notifications_table.php
category_product	2025_12_26_170100_create_product_category_and_brand_pivot_tables.php
brand_product	2025_12_26_170100_create_product_category_and_brand_pivot_tables.php
purchase_returns	2025_12_26_180000_create_purchase_returns_table.php
purchase_return_items	2025_12_26_180000_create_purchase_returns_table.php
secret_settings	2026_04_02_100001_create_secret_settings_table.php
product_prices	2026_05_16_000001_create_product_prices_table.php
privacy_mode_settings	2026_05_24_120000_create_privacy_mode_tables.php
privacy_mode_logs	2026_05_24_120000_create_privacy_mode_tables.php
chart_accounts	2026_06_05_000001_create_quickbooks_accounting_tables.php
account_transactions	2026_06_05_000001_create_quickbooks_accounting_tables.php
bank_accounts	2026_06_05_000001_create_quickbooks_accounting_tables.php
bank_reconciliations	2026_06_05_000001_create_quickbooks_accounting_tables.php
petty_cash_funds	2026_06_05_000001_create_quickbooks_accounting_tables.php
petty_cash_expenses	2026_06_05_000001_create_quickbooks_accounting_tables.php
stores	2026_06_05_000002_create_store_shipment_stock_tables.php
store_stocks	2026_06_05_000002_create_store_shipment_stock_tables.php
stock_shipments	2026_06_05_000002_create_store_shipment_stock_tables.php
stock_shipment_items	2026_06_05_000002_create_store_shipment_stock_tables.php
stock_shipment_allocations	2026_06_05_000002_create_store_shipment_stock_tables.php
store_stock_transfers	2026_06_05_000002_create_store_shipment_stock_tables.php
cheque_payments	2026_06_06_000001_create_cheque_payments_table.php
owner_equity_movements	2026_06_07_000002_create_owner_equity_movements_table.php
product_store_exclusions	2026_06_09_003857_create_product_store_exclusions_table.php

Table	Creation migration
store_transfers	2026_06_16_180717_create_store_transfers_table.php
store_user	2026_06_17_012839_create_store_user_table.php
tax_types	2026_07_25_000001_create_tax_management_tables.php
tax_settings	2026_07_25_000001_create_tax_management_tables.php
product_tax_settings	2026_07_25_000001_create_tax_management_tables.php
transaction_tax_lines	2026_07_25_000001_create_tax_management_tables.php
tax_ledger_entries	2026_07_25_000001_create_tax_management_tables.php
tax_payments	2026_07_25_000001_create_tax_management_tables.php
tax_adjustments	2026_07_25_000001_create_tax_management_tables.php
pre_orders	2026_08_18_000001_create_pre_order_module.php
pre_order_items	2026_08_18_000001_create_pre_order_module.php
pre_order_activities	2026_08_18_000001_create_pre_order_module.php

Appendix C. Domain service function inventory

These services centralize part of the business logic. The list includes public service methods; private helpers remain internal implementation details. Their presence does not mean every controller uses them consistently. Decimal/tax, payment posting, stock reservation and reversal rules must stay aligned with the approved workflows.

BulkPaymentAccountingService

Posts customer/supplier allocated payments and chooses cash/bank accounts.

Public functions: recordCustomerPayment, recordSupplierPayment

Source: app/Services/BulkPaymentAccountingService.php

ChequePaymentService

Passes/returns pending sale-linked cheques, performs automatic passing and synchronizes pre-order sale balances.

Public functions: pass, return, autoPassEligible

Source: app/Services/ChequePaymentService.php

DashboardVisibilityService

Loads privileged controls, parses amount ranges, filters records and scales selected displayed values.

Public functions: defaults, parseRangeInput, rangesToInputString, rangesFromControls, rangesForUser, isAmountInRanges, isSuperAdmin, configRaw, configForUser, save, maskByPercentage, profitValue, customerValue, hiddenProductIdsForUser, isProductHiddenForUser, hiddenSupplierIdsForUser, hiddenCustomerIdsForUser, hiddenSaleIdsForUser, hiddenStoreIdsForUser, isSupplierHiddenForUser, isCustomerHiddenForUser, isSaleHiddenForUser, isStoreHiddenForUser

Source: app/Services/DashboardVisibilityService.php

DecimalMath

Scaled integer arithmetic for precise tax values and rounding.

Public functions: parse, format, currency, multiply, percentage, roundDiv

Source: app/Services/DecimalMath.php

ExpenseAccountingService

Synchronizes or reverses normal expense postings and updates cash/bank/petty accounts.

Public functions: sync, reverse

Source: app/Services/ExpenseAccountingService.php

PreOrderImageService

Stores/removes vehicle images and resolves their public/PDF paths.

Public functions: store, delete

Source: app/Services/PreOrderImageService.php

PreOrderService

Saves pre-orders, links inventory, changes prices, controls statuses, manages payments and stock.

Public functions: save, unsyncProduct, syncProduct, changePrice, cancel, reopen, complete, addPayment, deletePayment, currentStock, currentSellingPrice

Source: app/Services/PreOrderService.php

PrivacyModeService

Determines session masking, permissions, page rules, invoice labels and toggle logs.

Public functions: getSettings, isEnabled, canToggle, isActiveForUser, setting, shouldMaskForCurrentPage, maskAmount, orderedInvoiceLabel, applyDailyInvoiceLabels, displayInvoiceNumber, logAction

Source: app/Services/PrivacyModeService.php

SalePaymentAccountingService

Records/reverses sale/pre-order payments, holds/clears cheques and updates selected accounts.

Public functions: recordSalePayment, recordPreOrderPayment, recordChequeHold, recordPreOrderChequeHold, recordChequePass, reversePayment, reversePreOrderPayment, reverseCheque, reverseChequeHold, reversePreOrderChequeHold

Source: app/Services/SalePaymentAccountingService.php

SaleRecalculationService

Recalculates original sale financial totals/status after returns.

Public functions: recalculateSaleFinancials, returnedQtyBySaleItem

Source: app/Services/SaleRecalculationService.php

SmsService

Sends SMS via configured HTTP provider methods; bulk sending reports success count.

Public functions: __construct, sendBulk, send, testConnection

Source: app/Services/SmsService.php

StockAlertService

Creates internal low-stock notifications after supported stock checks.

Public functions: check

Source: app/Services/StockAlertService.php

TaxCalculationService

Applies product tax rules, discounts, inclusive/exclusive VAT and invoice rounding.

Public functions: productRules, calculateLine, calculateInvoice, settingsForDate

Source: app/Services/TaxCalculationService.php

TaxInvoiceNumberService

Checks official invoice eligibility and issues a unique sequence number.

Public functions: issue, assertEligible

Source: app/Services/TaxInvoiceNumberService.php

TaxPostingService

Snapshots quotations, posts sale/purchase tax and records reversals/accounting.

Public functions: snapshotQuotation, postSale, postPurchase, reverse

Source: app/Services/TaxPostingService.php

TaxReturnService

Creates return tax records using original transaction snapshots.

Public functions: recordSaleReturn, recordPurchaseReturn

Source: app/Services/TaxReturnService.php

WhatsAppService

Generates links for manual WhatsApp Web sending.

Public functions: sendBulk, generateLink

Source: app/Services/WhatsAppService.php

Support, middleware and command entry points

Source	Entry points
app/Support/DatabaseAutoIncrementRepair.php	repairPrimaryId
app/Support/PublicStorageSync.php	syncFile, removeFile, linkAndSyncAll
app/Support/SecretPos.php	salesHiddenRanges, purchaseHiddenRanges, isHidden, isSaleHidden, isPurchaseHidden, mask, maskForSale, currencyMask, currencyMaskForSale, excludeHiddenRanges, excludeHiddenSaleRanges, excludeHiddenPurchaseRanges
app/Http/Middleware/CheckPrivacyMode.php	handle
app/Http/Middleware/EnsureSuperAdmin.php	handle
app/Http/Middleware/LockdownIfDbError.php	handle
app/Http/Middleware/RequirePermission.php	handle
app/Console/Commands/AutoPassChequePayments.php	handle CLI: cheques:auto-pass
app/Console/Commands/GenerateSampleData.php	handle CLI: app:generate-sample-data {--clean : Remove existing sample data first}
app/Console/Commands/SendMonthlyDueReminders.php	handle CLI: monthly:due-reminders

Appendix D. Source map, maintenance commands and glossary

D.1 Source map

Location	Responsibility
routes/web.php	Browser routes, permissions, AJAX endpoints and maintenance closures
routes/console.php; app/Console	Commands and legacy schedule declarations
app/Http/Controllers	Module orchestration, validation, views and exports
app/Services	Tax arithmetic/posting, payments, pre-orders, alerts, privacy and communication
app/Models	Records, relationships, casts, numbering and computed balances
app/Http/Middleware	Permission gate, super-admin restriction, privacy view state, force-error setting
app/Support	Visibility helpers, auto-increment repair and public storage synchronization
app/Imports; app/Exports	Product template/import and tax spreadsheet export
database/migrations	Schema creation and evolution
database/seeders	Reference/demo data, seeded roles/users and permission updates
resources/views	Blade UI, printable invoices, reports and PDFs
config; bootstrap	Framework, database, mail, filesystem, queue and session configuration
tests	Existing unit/feature verification

README_VEHICLEPOS.md, CLIENT_SYSTEM_FUNCTIONS.txt, PROJECT_SUMMARY.md, INSTALLATION.md, QUICK_REFERENCE.md, UNIT_SELECTION_GUIDE.md and TEXTIT_BIZ_INTEGRATION.md are useful background but older than several implemented modules. This document prioritizes current code when descriptions disagree. ReportController_recovered.php is not registered in the route inventory; SecretPosAdminController.php is empty. Standalone fix/test/seed/clear scripts are maintenance artifacts, not ordinary user functions.

D.2 Command reference

Command	Purpose and condition
php artisan route:list	Inspect actual registered endpoints
php artisan schedule:list	Confirm schedules are wired and show next-run information
php artisan migrate:status	Inspect migration state before deployment
php artisan migrate --force	Apply approved production migrations after backup
php artisan storage:link	Set up public storage access
php artisan optimize:clear	Clear cached configuration/routes/views during controlled troubleshooting
npm run build	Build deployable frontend assets
php vendor/bin/phpunit --do-not-cache-result	Run the existing test suite using isolated testing configuration
php artisan monthly:due-reminders	Dispatch configured reminders; can send real messages
php artisan cheques:auto-pass	Mutate eligible cheque status/financial records
php artisan schedule:run	Run registered due schedules; configure after schedule wiring is fixed

GenerateSampleData also exists; its exact signature and options are in app/Console/Commands/GenerateSampleData.php. Demonstration seed/clear scripts and DatabaseSeeder should be confined to controlled non-production use or a separately approved data process.

D.3 Glossary

Term	Meaning in this application
SKU	Unique internal product identifier
GRN	Goods received note identifier on a stock shipment
Price option	A product cost/selling-price pair with its own stock quantity
Store stock	Product availability at a specific branch, optionally tied to a price option
Held cheque	Pending cheque amount kept separate from cleared payment
Advance	Generic customer credit available for later invoice settlement
Pre-order deposit	Payment attached to an order before it becomes a completed sale
COGS	Cost of goods sold; current reports use present product cost
Input/output VAT	Purchase-side/sales-side tax recorded by the tax module
Tax snapshot	Historical calculation/settings data saved with a transaction
Privacy mask	Session/page-specific display modification
Visibility control	Privileged hiding/scaling/filtering of selected records or figures
Reversal	An operation undoing financial/tax/stock effects; implementation varies by module

Document owner should update this PDF after resolving the findings or changing business rules. Keep the accepted workflow decisions and validation results alongside each release.